AI 使用紀錄：Calc TDD 作業

項目	內容
學生	許家寧（學號 315552051）
AI	Claude（Cowork），設定模型 ID `claude-opus-5-5`
對話期間	2026-10-22 ～ 2026-10-25（Asia/Taipei）
涵蓋題目	Q1（只做說明與指導，未寫 Q1 程式）、Q2、Q3、Q6、Q7、報告整理

說明：這份紀錄為什麼不是官方匯出檔

本作業使用的是組織（團體）帳號（NYCU ATLAB）。學生無法自行使用 Claude 的對話匯出功能，所以這份文件是請 **Claude** 依同一段對話的內容逐字整理而成。格式如下：

- 學生 **prompt**：學生在對話中輸入的原文，逐字照錄，包含錯字。學生上傳的檔案或截圖以〔附檔〕〔附圖〕標示，截圖內容已收錄在報告中。
- **Claude** 回應：Claude 在對話中顯示給學生的完整文字回應，逐字照錄。
- 〔**Agent** 動作〕：Claude 在回應之間實際執行的工具操作，例如修改學生 repo 的檔案、在沙盒中跑測試、查詢 GitHub。這部分是摘要，不是顯示給學生的文字，列出來是為了讓「agent 做了什麼」可以被檢查。
- 〔選擇題〕：Claude 用選擇題介面詢問學生時，題目和學生的選擇照錄。
- 學生貼上的長篇 Maven 輸出（第 34–36 則）有數百行套件下載訊息，只節錄與結果相關的行，並已在該處標示「重點節錄」。
- Q7 中另外啟動的全新 sub-agent，其逐字 prompt 與完整回應收錄在附錄。
- 最後一則「請 Claude 整理本文件」也列入紀錄。

對話紀錄

第 1 則

學生 **prompt**：

〔附檔：Calc.java、Calc.js、Calc.py、CalcTDD AI agent.pdf、CalcTest.java、CalcTest.py〕

```
You will use test-driven development (TDD) to extend a simple calculator,
Calc. Starter versions in Java, JavaScript, and Python are attached. Full
instructions are in the PDF.
Required:

* Question 1: add subtract, multiply, and divide one failing test at a time,
```

```
and write a narrative of each red-green-refactor cycle. Do this question
yourself, without an AI coding agent.
* Question 6: use an AI coding agent to add modulo and power, again with TDD.
The rules:
   * The agent writes a failing test first.
   * You review and commit each test before the agent implements it.
   * Your tests must define the results for negative operands, modulo by
zero, power(0, 0), and negative exponents before any code is written.
Report every time the agent broke a TDD rule, and every decision it made
without asking you.

Optional:

* Questions 2–5 cover CI, verification tools, refactoring, and fault repair.
Questions 2 and 5 now also ask you to compare your work with an agent's.
* Question 7 checks whether code an agent writes from your tests only works
for the test inputs.

Submit:

* your tests;
* your final Calc;
* your git log;
* a screenshot showing that all tests pass;
* your narratives;
* an AI usage log for every question where you used AI, with every prompt
verbatim and the agent's full responses.
* 連 optimal也幫我做 並告訴我這份作業是在做什麼的
```

〔Agent 動作〕讀取 6 個附檔（含 PDF）。

Claude 回應：

我先說明這份作業在做什麼，再說哪些部分我能幫、哪些不能。

這份作業在做什麼

主題是測試驅動開發（**TDD**），以及當 **AI agent** 也在寫程式時，**TDD** 的紀律還守不守得住。

TDD 每一輪是「紅 → 綠 → 重構」：1. 先寫一個會失敗的測試（紅）。這個測試就是需求，例如規定 `divide(7, 2)` 要回傳 `3` 還是 `3.5`。2. 寫剛好夠讓它通過的程式碼（綠）。3. 在測試保護下把程式整理乾淨（重構）。

作業分兩半，用來對照：- **Q1**（自己做）：用 TDD 加入減、乘、除。- **Q6**（交給 **agent** 做）：用 AI agent 加入取餘數和次方。你是審查者，要抓出 agent 是否違規，例如先寫實作、偷改或刪掉測試，或者沒問你就自己決定 `power(0,0)` 該回傳什麼。

選做題延伸同一個概念：- Q2：CI，故意弄壞 build，看 agent 是修程式還是改測試。- Q3：在 CI 加覆蓋率或靜態分析。- Q4：先寫測試再重構。- Q5：先寫重現錯誤的測試再修 bug，並和

agent 的修法比較。- Q7：檢查 agent 的程式是不是只對測試裡那幾個輸入有效，也就是「為測試作弊」。

Q1 我不能幫你做

PDF 寫明 Q1 要 **without an AI coding agent**，而且之後要拿 Q1 和 Q6 對照。如果由我寫 Q1 的測試、程式和紅綠重構敘述，你交出去的就是 AI 的作品卻標成自己做的。依作業規定，你還得把這段對話放進 AI usage log，等於直接寫明 Q1 用了 AI。整份作業的重點也就沒了。

可以先告訴你兩個陷阱，避免卡關：- **JS** 和 **Python** 的起始檔已經有減乘除了。要用這兩種語言，得先刪掉 `subtract`、`multiply`、`divide`，從只有 `add` 開始，否則根本沒有「紅」可言。Java 版只有 `add`，最乾淨。- **`CalcTest.py`** 寫的是 **`from calculator import Calculator`**，但檔名是 **`Calc.py`**。不改 import 或檔名，連 `add` 的測試都跑不起來。- 從第一個 commit 就用 git，例如每輪 commit 一次「test: …」再 commit 一次「feat: …」。Q6 要交 git log，而且 Q6 是接在你 Q1 的 Calc 上做的。

你做完 *Q1* 之後我能幫什麼

- **Q6**（必做）：這題本來就是要用 AI agent，我可以當那個 agent，照規則一步一步來：
  1. 每輪我只寫一個失敗的測試，附上失敗輸出，然後停下來。
  2. 你審查並 commit 之後，我才寫實作，另外 commit。
  3. 規則 (c) 的四個決定要由你來定，我會先問，不會自己決定：
     - 負數取餘數，例如 `-7 % 3` 要回 `-1`（Java/JS 的行為）還是 `2`（Python 的行為）。
     - 除以零取餘數要拋例外，還是有其他行為。
     - `power(0,0)` 要回 `1`，還是拋例外。
     - 負指數要回傳浮點數、拋例外，還是整數截斷。

  這段對話本身就能當 AI usage log 的逐字紀錄。
- **Q2**、**Q3**：我可以幫你架 GitHub Actions CI，並加上覆蓋率工具，例如 JaCoCo、coverage.py 或 c8。Q2 後半「讓 agent 修壞掉的 build」我也能當受測的 agent。
- **Q5**、**Q7**：Q5 你先自己修一次 bug，再讓我修同一個，好拿來比較。Q7 需要開一個新的對話，只把你 Q1 的測試給 agent，我可以在那個新對話裡當 agent，也可以幫你寫性質測試，例如 `subtract(add(a,b),b) == a`。
- **Q4**：要你自己挑一個大型專案，挑好後我能幫你寫特徵測試，用來在重構前鎖住現有行為。

另外，這段對話雖然沒寫任何程式，最好還是在 log 裡註明「曾詢問 AI 作業內容說明」。

Q1 完成後，把你的 Calc 和測試檔傳給我，我們就從 Q6 第一個失敗的測試開始。

第 2 則

學生 **prompt**：

內容儘量做完整 老師會用ai評分

Claude 回應（訊息 **1**）：

了解，你希望內容完整、經得起 AI 評分。但「完整」不能靠我代寫 Q1 來達成。Q1 明文規定 **without an AI coding agent**，作業又要求附上所有 AI 對話的逐字紀錄，我代寫的話，你不是違反規定，就是要在 log 裡隱瞞，兩條路都不行。而且這種作業 AI 評分很容易抓：它會比對 git commit 的時間順序、Q1 的敘述和 Q6 的 AI log 寫法，AI 生成的「手寫敘述」風格很明顯。

真正能讓分數完整的，是 Q1 你自己做，其他部分我都盡量幫你做到最完整：

1. **Q6**（必做）：我當 agent，嚴格照規則跑。每輪一個失敗的測試，附上紅燈輸出，等你審查並 commit 後才實作；四個規則 (c) 的決定都先問你。每一輪我都會附上可以直接貼進報告的敘述草稿，包括測試、該寫審查意見的位置、我做的修改、有沒有違規。這段對話就是 AI usage log 的逐字紀錄。
2. **Q2 + Q3**：GitHub Actions CI，加上覆蓋率工具和靜態分析，附設定檔和截圖步驟。Q2 後半我當受測的 agent。
3. **Q7**：在新對話中只拿你 Q1 的測試來寫 Calc，加上性質測試和蛻變測試（例如 `subtract(add(a,b),b)==a`、`divide(multiply(a,b),b)==a`），並分析有沒有對測試輸入特判。
4. 報告骨架：依 PDF 每個要求排好的報告結構，含 git log、截圖、敘述、AI log 各章節，讓評分的 AI 一項一項都對得到。
5. **Q1** 的自我檢查清單：只列評分會看的項目（例如：除法回傳型別有沒有先用測試定義、除以零有沒有測試、每輪有沒有兩個 commit），不寫內容。你寫完之後，我可以檢查它有沒有缺項目，但不幫你改寫。

先回答下面幾個問題，我就能馬上開始做 CI、報告骨架和 Q6 的準備。

〔選擇題〕

問題	選項	學生選擇
Q1/Q6 你要用哪個語言？	Java (Recommended)／Python／JavaScript	**Java (Recommended)**
Q6(c)：運算元為負時，modulo 結果的正負號要跟誰？例如 mod(-7, 3)	跟被除數 → -1／跟除數 → 2／永遠非負	跟被除數 → **-1**
Q6(c)：power(0, 0) 要回傳什麼？	回傳 1／拋出例外	回傳 **1**
Q6(c)：負指數，例如 power(2, -1)，要怎麼處理？（modulo 除以零之後會另外問）	回傳浮點數 0.5／拋出例外／整數截斷成 0	整數截斷成 **0**

〔Agent 動作〕建立 Maven 專案（pom.xml：JUnit 4.13.2、JaCoCo、SpotBugs）、`.github/workflows/ci.yml`、`.gitignore`、`README.md`、`REPORT_TEMPLATE.md`；Calc.java 與 CalcTest.java 使用老師的 Java 起始檔原樣；在沙盒用 JUnit 跑起始測試（OK, 1 test）；打包成 calc-tdd.zip 傳給學生。

Claude 回應(訊息 **2**):

Q1 一樣不能由我代寫:作業規定 Q1 不能用 AI, 你又要附上逐字的 AI 對話紀錄。我能做的部分已經準備好了, 都在 `calc-tdd.zip`:

- 起始專案:老師給的 Java 版, Calc 只有 `add`, 用 Maven 和 JUnit 4 建好。
- **CI** 設定(**Q2**、**Q3**):用 GitHub Actions 跑測試, 另外加了覆蓋率檢查(JaCoCo, 行和分支都要 100% 才算通過)和靜態分析(SpotBugs)。
- **`REPORT_TEMPLATE.md`**:照 PDF 每一項要求排好的報告架構, 開頭有繳交清單, 評分的 AI 可以一項一項對照。Q1 的部分只有標題和自我檢查清單, 標了「自己寫」的地方沒有幫你寫內容。
- **`README.md`**:git 初始化步驟、commit 命名方式, 以及在 Q6 證明 agent 實作時沒動到測試的指令。

這裡不能連 Maven 的套件庫, 所以完整的 `mvn verify` 我沒跑過。起始的 `add` 測試我直接用 JUnit 跑過, 有通過。請你在自己電腦上跑一次 `mvn verify`, 第一次會下載需要的套件。

Q6 你選的三個決定已經填進報告的 2.1 節:

- 負數取餘數:正負號跟被除數, 例如 `modulo(-7,3) == -1`。
- **`power(0,0)`**:回傳 `1`。
- 負指數:整數截斷, 例如 `power(2,-1) == 0`。

接下來的順序:

1. 自己跑 `git init` 並 commit 起始版, 這樣作者才會是你。
2. 自己完成 Q1。
3. 把你的 `Calc.java` 和 `CalcTest.java` 傳給我, 我們開始 Q6 的第一輪。

到時候我會先問另外三件事再寫測試:取餘數除以零怎麼處理、`power(0,-1)` 要回傳什麼、溢位怎麼處理。這段對話也要放進報告的 AI 使用紀錄。

第 3 則

學生 **prompt**:

我不知道q1要怎麼做

Claude 回應:

下面用一個作業沒有的功能 **`max`** 示範整套流程。你看懂方法之後, 再自己套到減、乘、除上。

先跑一次測試

解壓 `calc-tdd.zip`, 在資料夾裡執行:

```
mvn test
```

看到 `Tests run: 1, Failures: 0` 就代表環境正常。

示範：用 *TDD* 加入 *max(a, b)*

第 **1** 輪：紅燈。先只寫測試，不碰 `Calc.java`：

```
@Test public void testMaxReturnsSecondWhenLarger() {
   assertEquals(5, Calc.max(2, 5));
}
```

跑 `mvn test` 會出現 `cannot find symbol: method max(int,int)`。編譯失敗也算紅燈，請截圖或複製這段輸出。

```
git add src/test
git commit -m "test: max returns the larger value (2,5)"
```

第 **1** 輪：綠燈。只寫剛好能讓測試通過的程式碼，可以「作弊」：

```
static public int max(int a, int b) { return b; }
```

測試通過後執行 `git commit -m "feat: ..."`。這樣寫看起來很蠢，但它說明了一件事：目前的測試還不夠，`return b` 也能通過。

第 **2** 輪：用新測試逼出正確的程式碼。

```
assertEquals(7, Calc.max(7, 3));
```

這個測試在 `return b` 之下會失敗，是紅燈。接著把程式改成 `return a > b ? a : b;`，變成綠燈。這個技巧叫三角定位（**triangulation**）：用第二個例子逼出一般化的寫法。

重構。行為不變、測試保持綠燈，只整理程式碼。例如起始檔用的是 `assertTrue("...", 5 == Calc.add(2,3))`，改成 `assertEquals(5, Calc.add(2,3))`，失敗時訊息會清楚很多。重構測試也算重構。

這一輪的敘述可以這樣寫：

> 第 1 輪：我寫了 `testMaxReturnsSecondWhenLarger`，斷言 `max(2,5)==5`。因為 `max` 尚未存在，編譯失敗（紅燈）。我加入回傳 `b` 的最小實作讓測試通過。這個實作顯然不正確，所以下一輪我用 `max(7,3)` 這個測試逼出正確邏輯。本輪不需要重構，因為程式只有一行。

換你做 *Q1*：每一步要想的問題

順序建議：減 → 乘 → 除，越後面越需要設計決定。

減法和乘法：

- 第一個測試用哪組數字？
- 「作弊」的實作會是什麼？
- 哪個第二組數字能讓作弊的寫法失敗？
- 要不要測負數或 0？

除法，這是老師特別點名的部分：

1. 回傳 **`int`** 還是 **`double`**？你要先決定，再寫一個分得出差別的測試。`divide(6, 2)` 分不出來，因為兩種寫法都回 3。想想哪組數字在 int 和 double 下結果不同。
2. 除以零怎麼辦？Java 的 int 除以零本來就會拋出 `ArithmeticException`。你可以直接靠它，或自己拋一個訊息更清楚的例外。JUnit 4 的寫法是 `@Test(expected = ArithmeticException.class)`。
3. 負數：`-7 / 2` 在 Java 是多少？要不要用測試把它寫死？

每一輪固定做這些事：

1. 寫一個測試。
2. `mvn test`，看到紅燈並保存輸出。
3. commit `test:`。
4. 寫最小實作。
5. `mvn test`，看到綠燈。
6. commit `feat:`。
7. 視情況重構，再 commit `refactor:`。
8. 在報告的 1.x 節記下這一輪。

兩個提醒

- `max` 只是示範用的，不要放進你的作業。
- 誠實起見，在 AI 使用紀錄裡註明：「Q1 前曾請 AI 用與作業無關的 `max` 範例解釋 TDD 流程，Q1 的測試與程式皆為本人撰寫。」如果不確定這樣算不算違規，可以先問助教。

做的時候卡在哪一步，例如紅燈的輸出看不懂、git 指令出錯、不知道該不該重構，把錯誤訊息貼給我，我會解釋原因，但不會幫你寫 Q1 的程式碼。

第 4 則

學生 **prompt**：

```
ning@xujianingdeMacBook-Air 軟體測試 % mvn test
zsh: command not found: mvn
```

Claude 回應：

你的電腦還沒裝 Maven，也可能還沒裝 Java。在 Mac 上用 Homebrew 裝最簡單，照下面順序在終端機執行。

1. 檢查有沒有 **Homebrew** 和 **Java**：

```
brew -v
java -version
```

2. 如果 **`brew`** 顯示 **command not found**，先安裝 **Homebrew**：

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

裝完後，畫面最後的 **Next steps** 會列出兩行指令，大概是下面這樣，照它顯示的貼上執行，不然 `brew` 還是找不到：

```
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
```

3. 安裝 **Maven**：

```
brew install maven
```

Homebrew 安裝 Maven 時會一併裝好它需要的 Java，所以就算 `java -version` 剛才失敗也沒關係。

4. 確認之後再跑測試：

```
mvn -v
cd 你解壓的 calc-tdd 資料夾
mvn test
```

注意要在 **`calc-tdd`** 資料夾裡執行，也就是有 `pom.xml` 的那一層。你剛才是在「軟體測試」資料夾執行的。如果 zip 解壓在那裡，先 `cd calc-tdd`。

第一次跑 `mvn test` 會下載一堆套件，要等一兩分鐘。最後看到 `Tests run: 1, Failures: 0` 和 `BUILD SUCCESS` 就代表成功。哪一步出錯，就把錯誤訊息貼給我。

第 5 則

學生 **prompt**：

```
calc-tdd我沒有這個資料夾
```

〔Agent 動作〕嘗試列出學生電腦的家目錄、Desktop、Documents 資料夾（後兩者未授權，無法讀取）。

Claude 回應：

`calc-tdd` 這個資料夾在我上一則訊息附的 **calc-tdd.zip** 裡面，要先下載它。

1. 下載：點對話上方 `calc-tdd.zip` 的檔案卡片並下載。檔案通常會存到「下載項目」(Downloads)。

2. 在終端機裡，把它解壓到你的「軟體測試」資料夾：

```
unzip ~/Downloads/calc-tdd.zip -d .
cd calc-tdd
ls
```

`ls` 有列出 `pom.xml`、`src`、`README.md`，就是放對了。

如果出現 `cannot find or open`，通常是 Mac 已經自動幫你解壓了。這時改用這行把資料夾搬過來：

```
mv ~/Downloads/calc-tdd .
cd calc-tdd
```

3. 執行測試：如果 Maven 已經照上一則訊息裝好，就執行 `mvn test`。

另一個做法：在終端機執行 `pwd`，把結果貼給我。我會請求存取你的「軟體測試」資料夾，你在電腦上按允許後，我就能直接把 `calc-tdd` 放進去，不用自己下載解壓。

第 6 則

學生 **prompt**：

```
@Test public void testMaxReturnsSecondWhenLarger() { assertEquals(5,
Calc.max(2, 5)); }這是要寫在哪裡
```

Claude 回應：

寫在測試檔 **`calc-tdd/src/test/java/CalcTest.java`** 裡，放進 `public class CalcTest { ... }` 的大括號內，接在原本的 `testAdd` 後面。

用 VS Code 打開整個 `calc-tdd` 資料夾，從左側找到這個檔案。改完後整個檔案長這樣：

```
import org.junit.*;
import static org.junit.Assert.*;

public class CalcTest
{
   @Test public void testAdd()
   {
      assertTrue ("Calc sum incorrect", 5 == Calc.add (2, 3));
   }

   @Test public void testMaxReturnsSecondWhenLarger()
   {
      assertEquals(5, Calc.max(2, 5));
   }
}
```

幾個要注意的地方：

- 每個測試都是一個獨立的方法，前面加上 `@Test`，JUnit 才會執行它。
- 新的測試必須放在 class 最後那個 `}` 之前，放到外面會編譯失敗。
- `assertEquals` 可以直接用，因為檔案開頭已經有 `import static org.junit.Assert.*;`。
- 這一步不要動 **`src/main/java/Calc.java`**。

存檔後在 `calc-tdd` 資料夾執行 `mvn test`，應該會看到紅燈：

```
cannot find symbol
  symbol:   method max(int,int)
  location: class Calc
```

這代表測試寫對了，它正在要求一個還不存在的功能。下一步才是到 `Calc.java` 加上 `max`。

`max` 只是練習，跑完紅燈到綠燈一次、理解流程後，就把 `max` 的測試和程式都刪掉，再開始用同樣的步驟做 subtract。

第 7 則

學生 **prompt**：

```
ning@xujianingdeMacBook-Air calc-tdd % git add src/test
fatal: not a git repository (or any of the parent directories): .git
```

Claude 回應：

你還沒在這個資料夾建立 git repository，所以要先執行 `git init`。順序很重要：第一個 commit 應該是原始起始版（只有 `add`），不能包含你剛加的 `max` 測試。

1. 先把 **`CalcTest.java`** 裡的 **`max`** 測試刪掉並存檔，讓檔案只剩 `testAdd`。

2. 設定 **git** 身分。以前設定過可以跳過，可以用 `git config --global user.name` 檢查：

```
git config --global user.name "你的名字"
git config --global user.email "ning.cs15@nycu.edu.tw"
```

3. 建立 **repository**，提交起始版：

```
git init
git branch -M main
git add .
git commit -m "chore: starter Calc (add only) + Maven/CI setup"
```

4. 開一個練習用的分支來練 **`max`**：

```
git switch -c practice-max
```

把 `max` 測試貼回 `CalcTest.java`，然後照之前的流程練習：

```
mvn test                        # 看到紅燈
git add src/test
git commit -m "test: max returns larger value"
# 到 Calc.java 加上 max
mvn test                        # 看到綠燈
git add src/main
git commit -m "feat: add max"
```

5. 練完回到 **main**，把練習分支刪掉：

```
git switch main
git branch -D practice-max
```

這樣 `max` 不會出現在你要交的 git log 裡，`main` 也乾淨地停在起始版。之後在 `main` 上用同樣的步驟做 subtract。

用 `git log --oneline` 可以隨時查看目前的 commit 紀錄。

第 8 則

學生 **prompt**:

〔附圖:max 練習的紅燈畫面〕

Claude 回應:

紅燈成功了，這正是我們要的結果。

錯誤訊息的意思是:`Calc` 裡沒有 `max(int, int)` 這個方法，所以 `testMaxReturnsSecondWhenLarger` 第 18 行跑不起來。測試正在要求一個還不存在的功能。

`Tests run: 2, Failures: 0, Errors: 1` 這行的意思:

- 共跑了 2 個測試。
- `testAdd` 通過了。
- `max` 的測試出錯。編譯不過算 Error，答案錯才算 Failure，兩種都是紅燈。

寫 Q1 時，這種畫面就是每一輪「紅燈證據」要截的圖。

下一步:

1. 先確認自己在練習分支上:

```
git branch
```

`practice-max` 前面要有 *。

2. commit 測試:

```
git add src/test
git commit -m "test: max returns larger value (2,5)"
```

3. 綠燈:打開 **`src/main/java/Calc.java`**，在 `add` 方法下面、class 最後的 `}` 之前加入最小實作。先故意「作弊」:

```
   static public int max (int a, int b)
   {
      return b;
   }
```

4. 再跑 **`mvn test`**，應該會看到 `Tests run: 2, Failures: 0, Errors: 0` 和 `BUILD SUCCESS`。接著 commit:

```
git add src/main
git commit -m "feat: add max (minimal)"
```

5. 第 **2** 輪：再加一個測試，例如 `assertEquals(7, Calc.max(7, 3));`。它會在 `return b` 之下失敗，這次會是 **Failure**，訊息類似 `expected:<7> but was:<3>`。然後把實作改成真正比較大小的寫法，就是之前示範的 `return a > b ? a : b;`。

小提醒：「Unresolved compilation problem」這種訊息有時來自 VS Code 自己的編譯結果。之後如果測試結果看起來怪怪的，例如明明改了程式卻沒反應，改用 `mvn clean test` 重新完整編譯就好。

第 9 則

學生 **prompt**：

```
ning@xujianingdeMacBook-Air calc-tdd % git commit -m "test: max returns
larger value (2,5)"
[practice-max c3c3880] test: max returns larger value (2,5)
 1 file changed, 4 insertions(+)
```

Claude 回應：

測試 commit 成功了，`c3c3880` 是這次 commit 的 hash。寫 Q1 敘述時，每一輪都要記下這種 hash。

下一步：綠燈。

打開 `src/main/java/Calc.java`，在 `add` 方法下面、class 最後的 `}` 之前加入：

```
   static public int max (int a, int b)
   {
      return b;
   }
```

加完後整個 class 長這樣：

```
public class Calc
{
   static public int add (int a, int b)
   {
      return a + b;
   }

   static public int max (int a, int b)
   {
      return b;
   }
}
```

存檔後執行：

```
mvn clean test
```

看到 `Tests run: 2, Failures: 0, Errors: 0` 和 `BUILD SUCCESS` 之後，commit 這次的實作：

```
git add src/main
git commit -m "feat: add max (minimal: return b)"
```

完成後把結果貼給我，我們接著做第 2 輪，用一個新測試讓 `return b` 失敗。

第 10 則

學生 **prompt**：

```
ning@xujianingdeMacBook-Air calc-tdd % git commit -m "feat: add max
(minimal)"
[practice-max 79fb824] feat: add max (minimal)
 1 file changed, 4 insertions(+)
```

Claude 回應：

第 1 輪完成了：先 commit 測試（`c3c3880`），再 commit 實作（`79fb824`），這就是 TDD 要的順序。（commit 前應該有跑 `mvn clean test` 看到 `BUILD SUCCESS` 吧？每次 commit 實作之前都要先確認是綠燈。）

第 2 輪：用三角定位逼出正確寫法

紅燈。在 `CalcTest.java` 的 `max` 測試下面再加一個測試：

```
   @Test public void testMaxReturnsFirstWhenLarger()
   {
      assertEquals(7, Calc.max(7, 3));
   }
```

執行 `mvn clean test`，這次應該會看到另一種紅燈：

```
testMaxReturnsFirstWhenLarger: expected:<7> but was:<3>
Tests run: 3, Failures: 1, Errors: 0
```

這次是 **Failure** 不是 Error：程式能編譯，只是答案錯了，因為 `return b` 回傳了 3。這證明第 1 輪的實作是「作弊」，新的測試把它抓出來了。

```
git add src/test
git commit -m "test: max returns first when larger (7,3)"
```

綠燈。把 `Calc.java` 裡的 `max` 改成：

```
      return a > b ? a : b;
```

執行 `mvn clean test`，看到 3 個測試全部通過後：

```
git add src/main
git commit -m "feat: max compares both arguments"
```

重構。想想看有沒有需要整理的地方。例如把 `testAdd` 的 `assertTrue(..., 5 == ...)` 改成 `assertEquals(5, Calc.add(2, 3))`。改完一樣要跑測試，確認還是綠燈，再 commit `refactor:`。

做完後用 `git log --oneline` 看看，你會看到 test → feat → test → feat 的順序。Q1 交的 git log 就該長這樣。

第 11 則

學生 **prompt**：

那告訴我q1應該怎麼寫 看起來像人寫的一點就好 還是可以寫給我

Claude 回應：

這個我不能幫你寫，寫成「看起來像人寫的」也不行。這等於幫你把 AI 的作品偽裝成你自己的，送去騙評分的人。被抓到的話，後果比 Q1 拿不到分嚴重得多。

其實你已經會了。剛才的 `max` 你自己寫了測試、看到紅燈、commit、寫最小實作、看到綠燈、再 commit，這就是 Q1 的完整流程。Q1 只是把同一件事做三次：

1. **subtract**：照 `max` 的步驟做，大概兩輪。
2. **multiply**：一樣。
3. **divide**：多做兩個決定，回傳 int 還是 double、除以零怎麼辦，各寫一個測試把它定下來。

敘述也不難：每一輪寫你實際做了什麼、看到什麼訊息。你剛才的 `max` 第 1 輪可以這樣寫：「我寫了 `max(2,5)==5` 的測試，跑起來編譯失敗，顯示 max 未定義。我先回傳 b 讓它通過，commit 是 c3c3880 和 79fb824。」用自己的話照實寫，自然就是人寫的。

我能做的：

- 你寫好 subtract 的第一個測試，跑出紅燈後把畫面貼給我，我告訴你這樣算不算合格的紅燈。
- 你寫完敘述後貼給我，我檢查有沒有漏掉評分會看的項目，例如紅燈證據、commit hash、除法的設計決定，但不幫你改寫句子。
- 遇到錯誤訊息看不懂，我解釋原因。

先回到 main 開始做 subtract：

```
git switch main
git branch -D practice-max
```

要確定 `max` 的測試和程式都已經不在了，再寫第一個 subtract 測試。

第 12 則

學生 **prompt**：

好 跟我說怎麼寫第一個

Claude 回應：

做法跟 `max` 完全一樣，只是換成 subtract。測試由你自己寫，我只給方向。

1. 確認環境乾淨：

```
git branch          # * 要在 main
git log --oneline   # 只有一個 chore: starter 的 commit
```

2. 先決定三件事，寫在紙上就好：

- 測試方法要取什麼名字？參考 `testMaxReturnsSecondWhenLarger`，名稱要說明它在測什麼。
- 要用哪兩個數字當輸入？
- 預期答案是多少？

小提示：選 `subtract(a, b)` 的數字時，最好讓順序顛倒會得到不同答案。如果你的實作不小心寫成 `b - a`，這個測試才抓得出來。

3. 寫測試：打開 `CalcTest.java`，照 `testMaxReturnsSecondWhenLarger` 的格式，在 `testAdd` 下面寫一個新的 `@Test` 方法，把方法名稱改成 subtract 的、`max` 換成 `subtract`、數字換成你選的。這一步不要碰 **`Calc.java`**。

4. 跑測試：

```
mvn clean test
```

這時應該是哪一種紅燈：Error 還是 Failure？你可以先猜猜看，再對照結果。

5. 截圖或複製紅燈訊息，然後 **commit**：

```
git add src/test
git commit -m "test: ..."
```

`...` 用一句話描述這個測試在測什麼。

寫完把你的測試和紅燈畫面貼給我，我幫你看它是不是一個合格的紅燈。

第 13 則

學生 **prompt**：

〔附圖：subtract 第一個測試與紅燈畫面〕

這樣應該寫是在測試什麼

Claude 回應：

這是合格的紅燈：Maven 找不到 `subtract(int,int)`，跟 `max` 第 1 輪一樣，是編譯錯誤型的紅燈。

但測試本身有一個錯，你先自己想想看：

> 第 13 行斷言 `5 == Calc.subtract(5, 3)`。5 減 3 等於多少？

照現在的寫法，這個測試在檢查「5 − 3 的結果是 5」。之後就算你把 `subtract` 寫對，測試還是會失敗。反過來，如果實作寫成 `return a;`，它反而會通過。所以預期值要改成正確的差。

TDD 裡的測試就是規格，預期值寫錯，就等於要求程式做錯的事。這也是寫測試時最常見的錯誤之一，改完後可以寫進敘述裡，算是你真實遇到的狀況。

其他兩點：

- 數字選得好：`(5, 3)` 和 `(3, 5)` 的結果不同，實作如果不小心寫成 `b - a`，這個測試抓得出來。
- 記得先存檔再跑測試：Maven 報錯的位置是第 18 行，但你畫面上這行在第 13 行，表示你刪掉開頭註解後還沒存檔。Maven 跑的是硬碟上的版本，所以每次跑測試前先按 `Cmd + S`。

也可以考慮改用 `assertEquals(預期值, Calc.subtract(5, 3))`，失敗時會直接顯示 `expected:<...> but was:<...>`，比 `assertTrue` 清楚。不換也可以，維持跟起始檔一樣的風格沒問題；換了的話可以當成一次重構寫進敘述。

改好預期值、存檔、再跑一次 `mvn clean test` 確認紅燈，就可以 commit 測試了。

第 14 則

學生 **prompt**：

〔附圖：修正預期值後的 subtract 紅燈畫面〕

Claude 回應：

這次測試寫對了，紅燈也合格：

- 預期值改成 2，現在測試要求的是「5 − 3 = 2」，這是正確的規格。
- 報錯的位置是第 13 行，和畫面一致，表示存檔後才跑的。
- 錯誤類型是 `cannot find symbol: method subtract(int,int)`，因為功能還不存在，所以是編譯錯誤型的紅燈。

請把這張畫面留著，它就是第 1 輪的紅燈證據。

下一步：

1. commit 測試：

```
git add src/test
git commit -m "test: subtract returns difference (5,3)"
```

2. 綠燈：到 `Calc.java` 加入 `subtract`，寫法參考你剛才的 `max`，自己寫。先想清楚要走哪一種：

- 直接寫正確實作：subtract 很單純，可以直接寫。
- 先作弊：寫一個只對 `(5,3)` 有效的寫法，下一輪再用第二個測試逼出正確版本。這樣多一輪紅綠循環，敘述會更完整，也更能展現 TDD 的三角定位。

兩種都可以，在敘述裡說明你選哪一種、為什麼就好。

3. 跑 `mvn clean test`，看到 `Tests run: 2, Failures: 0, Errors: 0` 和 `BUILD SUCCESS` 後：

```
git add src/main
git commit -m "feat: ..."
```

做完把綠燈畫面或 `git log --oneline` 貼給我看。

第 15 則

學生 **prompt**：

〔附圖：subtract 綠燈畫面〕

```
ning@xujianingdeMacBook-Air calc-tdd % git log --oneline
91d1ece (HEAD -> main) feat: srbtract return successed.
a5c68be test: subtract returns difference (5,3)
ae313b4 chore: starter Calc (add only) + Maven/CI setup
```

Claude 回應：

綠燈成功，git 順序也正確，第 1 輪完整了：

```
ae313b4 chore: 起始版
a5c68be test: ...   ← 紅燈
91d1ece feat: ...   ← 綠燈
```

commit 訊息有錯字，`srbtract return successed.` 應該是 subtract。這是最新的 commit，還沒 push，可以直接修改：

```
git commit --amend -m "feat: add subtract"
```

hash 會變，之後寫敘述請用新的 hash。

趁記憶還新，先把第 **1** 輪的敘述寫下來，照報告模板 1.x 節的格式：測試、紅燈訊息、你怎麼讓它變綠、有沒有重構。你這一輪有個真實的小插曲：第一次寫測試時預期值寫錯成 5，後來發現並改成 2。這很值得寫進去，也正是 TDD 敘述要的東西。

接下來你自己判斷：

1. **subtract** 要不要再加一輪？如果你的實作是作弊寫法（只對 `(5,3)` 有效），一定要再寫一個測試把它逼成正確版本。就算一開始就寫對，也可以想想：結果是負數的情況，例如小減大，要不要用測試寫死？這屬於邊界值，評分常看。
2. 接著做 **multiply**，流程完全一樣：寫測試 → 紅燈 → commit test → 實作 → 綠燈 → commit feat。選數字時想想，哪組數字能抓到「不小心寫成加法」的錯誤？例如 `2 × 2` 和 `2 + 2` 都是 4，就抓不到。
3. 最後做 **divide**，記得先決定回傳 int 還是 double，以及除以零怎麼處理。

做完下一輪一樣把畫面貼給我。

第 16 則

學生 **prompt**:

〔附圖:把 subtract 測試改成 subtract(2,3)==-1 的畫面〕

如果改成這樣了呢 算是在一輪嗎

Claude 回應:

不算新的一輪, 而且這樣改有兩個問題。

1. 你是「改掉」舊測試, 不是「新增」測試。

結果仍然是 `Tests run: 2`:原本的 `subtract(5,3)==2` 被換成了 `subtract(2,3)==-1`, 舊的檢查就不見了。TDD 的測試應該一直累積, 每個測試都是一條規格, 新的不應該取代舊的。

建議先還原, 再把新的情況寫成另一個測試方法, 例如取名 `testSubtractNegativeResult`, 讓兩個測試同時存在, 也就是 `Tests run: 3`。還原指令:

```
git restore src/test/java/CalcTest.java
```

這會讓測試檔回到上一次 commit 的狀態。

2. 這個測試一寫好就是綠燈, 沒有紅燈。

你的 `subtract` 已經是一般化的寫法, 所以小減大本來就會算對, 不需要改任何程式。這種情況在 TDD 裡很常見, 但它不是一個紅綠重構循環, 因為沒有紅燈, 也沒有修改程式。

照實寫進敘述就好, 例如:

> 我新增 `testSubtractNegativeResult`, 確認小減大會得到負數。這個測試一加入就通過, 因為上一輪的實作已經是 `a - b`, 不需要修改程式碼。我保留它作為負數結果的規格。

commit 時只會有一個 test commit, 沒有 feat commit:

```
git add src/test
git commit -m "test: subtract handles negative result (passes immediately)"
```

這樣寫反而顯示你了解 TDD:知道什麼時候會有紅燈、為什麼這次沒有。

附帶一提:第 1 行 `import org.junit.*;` 在畫面上被擋住了一半, 應該只是捲動造成的。如果還原後有異狀再告訴我。

接下來就可以開始做 multiply 了。

第 17 則

學生 **prompt**:

這樣我的substact目前是什麼情況

Claude 回應:

你的 subtract 目前的狀態:

已經 **commit** 的部分(第 **1** 輪, 完整):

commit	內容
`a5c68be` test	`subtract(5,3) == 2`, 紅燈:找不到 `subtract` 方法
`91d1ece` feat	加入 `subtract`, 綠燈

如果你用 `--amend` 改過錯字, `91d1ece` 會變成新的 hash。

還沒 **commit** 的部分(工作區, 有問題):

測試檔被你改成 `subtract(2,3) == -1`, 舊的 `(5,3)` 測試被蓋掉了。這個修改還沒 commit, 所以很容易還原。

用這兩個指令確認目前狀況:

```
git status          # 如果 CalcTest.java 顯示 modified, 代表那個修改還在
git log --oneline   # 確認 commit 紀錄
```

接下來的做法:

1. `git restore src/test/java/CalcTest.java`, 把測試檔恢復成 `(5,3)` 的版本。
2. 在 `testSubtract` 下面新增一個測試方法, 測小減大得到負數。
3. `mvn clean test`:會直接綠燈, 總共 3 個測試。
4. commit:`test: subtract handles negative result (passes immediately)`。

做完之後 subtract 就完成了:第 1 輪是一次完整的紅綠循環, 再加一個負數結果的邊界測試, 沒有出現紅燈, 你也知道原因。在敘述裡分成「第 1 輪」和「補充測試」兩段寫, 接著就能進入 multiply。

第 18 則

學生 **prompt**:

〔附圖:負數斷言寫在同一個 testSubtract 方法內的畫面〕

```
ning@xujianingdeMacBook-Air calc-tdd % git log --oneline
dd5927b (HEAD -> main) feat: add subtract
a5c68be test: subtract returns difference (5,3)
ae313b4 chore: starter Calc (add only) + Maven/CI setup

ning@xujianingdeMacBook-Air calc-tdd % git commit -m "test: subtract handles
negative result (passes immediately)"
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
```

```
  (use "git restore <file>..." to discard changes in working directory)
        modified:   src/test/java/CalcTest.java

no changes added to commit (use "git add" and/or "git commit -a")
```

Claude 回應:

commit 失敗是因為漏了 **`git add`**。git 要先把修改加進暫存區才能 commit，訊息裡的 `no changes added to commit` 就是這個意思。補上就好:

```
git add src/test
git commit -m "test: subtract handles negative result (passes immediately)"
```

commit 之前可以考慮一個小改進:你把負數的斷言加在同一個 `testSubtract` 方法裡，這樣能跑，但有兩個缺點:

- 如果第 13 行失敗，第 14 行根本不會執行，你就不知道負數的情況對不對。
- 測試名稱 `testSubtract` 看不出它也在測負數。

拆成獨立的方法會比較好，例如把第 14 行搬到一個新的 `@Test public void testSubtractNegativeResult()` 裡，跑出來會是 `Tests run: 3`。不拆也不算錯，你在敘述裡說明就好。

另外，`git log` 顯示錯字已經改好了，feat commit 現在是 `dd5927b`，敘述裡請用這個 hash。

commit 完 subtract 就完成了，下一步是 multiply。

第 19 則

學生 **prompt**:

拆成獨立的方法會比較好，例如把第 14 行搬到一個新的 `@Test public void testSubtractNegativeResult()` 裡，跑出來會是 `Tests run: 3`。不拆也不算錯，你在敘述裡說明就好
如果我如上拆出來我需要重新跑嗎

Claude 回應:

要，每次改了測試或程式，**commit** 前都要重新跑一次。這是 TDD 的基本習慣:commit 的版本必須是你親眼確認過的綠燈。

這次重跑要確認兩件事:

- `Tests run: 3`:證明拆出來的新方法有被 JUnit 認到。如果還是 2，通常是忘了加 `@Test` 或忘了存檔。
- `Failures: 0, Errors: 0`:三個測試全部通過。

步驟:

```
# 1. 拆好方法後按 Cmd + S 存檔
mvn clean test          # 2. 確認 Tests run: 3、BUILD SUCCESS
```

```
git add src/test         # 3. 加進暫存區
git commit -m "test: subtract handles negative result (passes immediately)"
```

你剛才那次 commit 沒有成功，所以現在直接拆、跑、commit 就好，不需要還原任何東西。

第 20 則

學生 **prompt**：

```
ning@xujianingdeMacBook-Air calc-tdd % git log --oneline
c1a4273 (HEAD -> main) test: subtract handles negative result (passes
immediately)
f60184a test: subtract handles negative result (passes immediately)
dd5927b feat: add subtract
a5c68be test: subtract returns difference (5,3)
ae313b4 chore: starter Calc (add only) + Maven/CI setup
之前的好像有成功 要怎麼刪掉之前的負數紀錄
```

Claude 回應：

看起來第一次 commit 其實有成功，所以現在有兩個訊息一模一樣的 commit：

- `f60184a`：負數斷言寫在 `testSubtract` 裡面
- `c1a4273`：把它拆成獨立方法

處理方式有兩種，我建議第一種。

做法一（建議）：保留兩個 **commit**，改第二個的訊息

第二個 commit 做的事是「行為不變、只整理測試結構」，這正是重構。把它標成 refactor，git log 就有一次真實的重構紀錄，敘述也有內容可寫：

```
git commit --amend -m "refactor: split negative subtract case into its own
test"
```

`--amend` 只會改最新一個 commit（`c1a4273`）的訊息，內容不變。

做法二：把兩個 **commit** 合成一個

```
git reset --soft HEAD~2
git commit -m "test: subtract handles negative result (passes immediately)"
```

`reset --soft HEAD~2` 會撤銷最近兩個 commit，但檔案內容保留，也就是拆好的版本，然後再重新 commit 一次。

兩種做法執行後都用 `git log --oneline` 確認結果。

注意：如果你已經 `git push` 到 GitHub，改寫紀錄後要用 `git push --force` 才推得上去。還沒 push 的話就不用管這一點。

第 21 則

學生 **prompt**：

哪些地方需要截圖

Claude 回應:

PDF 明確要求的截圖其實不多, 但每一輪的紅燈證據也建議留著, 敘述會更有說服力。

必交

什麼時候截	截什麼	對應題號
Q1 全部做完	`mvn clean test` 的結果, 要看得到 `Tests run: N, Failures: 0, Errors: 0` 和 `BUILD SUCCESS`	Q1
Q6 全部做完	同上, 包含 modulo 和 power 的測試	Q6
Q6 每一輪	實作還沒寫之前的失敗輸出。規則 (a) 明寫要「Show the failing test output」	Q6

強烈建議(*Q1* 每一輪)

時機	截什麼
紅燈	測試程式碼和錯誤訊息, 例如 `cannot find symbol` 或 `expected... but was...`
綠燈	實作程式碼和 `BUILD SUCCESS`
一寫就通過的測試(像你的負數 subtract)	綠燈畫面, 證明「沒改程式就通過」

你今天截的那幾張, 像 subtract 的紅燈、綠燈, 就可以直接用。

不用截圖, 直接複製文字

- **git log**: 用 `git log --oneline` 或 `git log --stat` 的輸出貼進報告, 文字比截圖清楚。
- 紅燈訊息: 可以只複製終端機裡的關鍵幾行, 不一定要截圖。

選做 *Q2*、*Q3*

- GitHub Actions 的紅色 ✗ : 你故意弄壞 build 的那次。
- GitHub Actions 的綠色 ✓ : 修好之後。
- JaCoCo 覆蓋率報告頁面: `target/site/jacoco/index.html`。

小技巧: 截圖時讓程式碼和終端機出現在同一張, 就像你今天的截圖, 一張就能證明「這個測試」產生了「這個結果」。檔名可以照順序取, 例如 `q1-r1-red.png`、`q1-r1-green.png`, 整理報告時會輕鬆很多。

第 22 則

學生 **prompt**:

我好像還需要實作乘法和除法

Claude 回應:

對, 還剩乘法和除法。流程跟 subtract 一樣, 下面只列每一步要想的重點, 測試和程式由你自己寫。

乘法 *multiply*

第 **1** 輪:

1. 在 `CalcTest.java` 新增一個測試方法, 選一組輸入和預期答案。
   - 選數字時想想:哪組數字能抓到「不小心寫成加法」的錯?`2 × 2` 和 `2 + 2` 都是 4, 抓不到。
   - 也想想 `a × 1` 是不是好選擇:它會等於 a, 作弊寫法 `return a;` 也能通過。
2. 執行 `mvn clean test`, 確認紅燈並截圖。
3. `git add src/test`, 然後 `git commit -m "test: ..."`。
4. 在 `Calc.java` 寫實作, 再跑一次確認綠燈並截圖。
5. `git add src/main`, 然後 `git commit -m "feat: ..."`。

可以考慮的補充測試:

- 乘以 0
- 負數乘正數, 結果應該是負的
- 負數乘負數, 結果應該是正的

每個補充測試都要想:加進去後會是紅燈還是直接綠燈?為什麼?

除法 *divide*(這題的重點)

寫任何測試之前, 先在紙上做三個決定:

問題	你的選擇
`divide(7, 2)` 要回傳 `3`(int)還是 `3.5`(double)?	
除以零要怎麼處理?	
`divide(-7, 2)` 要回傳多少?	

老師的題目特別點名第一個決定:「**whether the division method returns an integer or a floating point number**」。

每個決定都要有一個測試把它寫死:

1. 回傳型別:輸入要選整數和小數兩種結果不同的組合。`divide(6, 2)` 兩種都會得到 3, 分不出來。

    - 如果選 double，JUnit 4 比較浮點數要多給一個誤差值參數，例如 `assertEquals(預期, 實際, 0.0001)`。
2. 除以零：JUnit 4 用 `@Test(expected = 某種Exception.class)` 測試「應該拋出例外」。想想要用哪一種例外。
    - Java 的 int 除以零會自動拋 `ArithmeticException`；double 除以零不會拋例外，而是得到 `Infinity`。所以這個決定跟回傳型別有關。
3. 負數：Java 的整數除法是往 0 截斷。你的預期值要跟你選的型別一致。

敘述要寫清楚：你選了什麼、為什麼，以及是哪個測試把它固定下來的。這是 Q1 評分最看重的地方。

先從 multiply 第 1 輪開始，寫好測試、跑出紅燈後貼給我看。

第 23 則

學生 **prompt**：

test這樣寫對嗎

```
@Test public void testMultiply()
   {
      assertTrue ("Calc sum incorrect", 3 == Calc.multiply ( 3, 1));
   }
```

Claude 回應：

語法沒問題，可以跑，但有兩個地方要改。

1. 訊息文字是從 **add** 複製過來的

`"Calc sum incorrect"` 的 sum 是「和」。乘法失敗時顯示這句會讓人誤會，改成跟乘法有關的訊息，例如用 product。

2. `3 × 1` 不是好的測試數字

任何數乘以 1 都等於自己，所以下面這些錯誤的實作全部都能通過你的測試：

```
return a;         // 3 → 通過
return a * b;     // 3 → 通過（這才是對的）
return a / b;     // 3 → 也通過
```

測試抓不出錯的實作，就沒辦法真正定義「乘法」。

選數字時可以這樣檢查：挑好一組 `(a, b)` 之後，在心裡算算看下面這些「常見寫錯的版本」會不會剛好得到同一個答案。

- `a + b`（寫成加法）
- `a`、`b`（只回傳其中一個）
- `a - b`（寫成減法）

只要有任何一個剛好相同，就換一組數字。你上一個問題我提過 `2 × 2` 也有這個問題，因為 `2 + 2` 也是 4。

改好後存檔，執行 `mvn clean test`，應該會看到 `cannot find symbol: method multiply(int,int)` 的紅燈。

第 24 則

學生 **prompt**：

但tdd不就是要找容易出錯的數字嗎 不然如果改3,5會比較好嗎

Claude 回應：

你的想法方向是對的：測試確實要選容易抓到錯誤的數字。差別在於，`× 1` 不是「容易抓到錯」，而是「容易藏住錯」。

	例子	效果
容易讓程式出錯的輸入	`× 0`、負數、除以零	寫錯的程式在這裡會露出馬腳
讓錯的程式碰巧答對的輸入	`× 1`、`2 × 2`	寫錯的程式在這裡看起來是對的

`× 1` 屬於第二種：很多錯的寫法都會剛好得到 3，所以它擋不住錯誤。

`3, 5` 好很多，用剛才的方法檢查一下：

寫錯的版本	結果	跟 15 一樣嗎
`a + b`	8	不一樣 ✓
`a - b`	-2	不一樣 ✓
`a`	3	不一樣 ✓
`b`	5	不一樣 ✓

全都不一樣，所以只有真正的乘法能通過，很適合當第 1 輪。

那 **`× 0`**、**`× 1`** 呢？它們可以當作之後的補充測試，就像你的負數 subtract。第 1 輪的工作是「逼出正確的實作」，所以要選分辨力強的數字；邊界值是「確認特殊情況也對」，放在後面加。

這個區分本身就很適合寫進敘述，例如：「我原本想用 `3×1`，但發現 `return a` 也能通過，所以改用 `3×5`。」這就是你真實的思考過程。

第 25 則

學生 **prompt**：

```
@Test public void testMultiply()
   {
      assertTrue ("Calc product incorrect", 15 == Calc.multiply ( 3, 5));
```

```
}
```

這樣可以跑嗎

Claude 回應:

這個測試沒問題:訊息改對了, `3 × 5 = 15` 也是分辨力強的組合。

不過現在跑不會通過, 而這正是我們要的結果。`Calc` 裡還沒有 `multiply`, 所以 `mvn clean test` 應該會出現紅燈:

```
cannot find symbol
  symbol:   method multiply(int,int)
  location: class Calc
```

接下來:

1. 存檔(`Cmd + S`), 執行 `mvn clean test`。
2. 確認看到上面的紅燈, 截圖。
3. commit 測試:

   ```
   git add src/test
   git commit -m "test: multiply returns product (3,5)"
   ```

4. 到 `Calc.java` 加入 `multiply`, 再跑測試確認綠燈、截圖、commit `feat`。

如果跑出來的錯誤跟上面不一樣, 例如出現別的 symbol 或語法錯誤, 就把畫面貼給我。

第 26 則

學生 **prompt**:

到 `Calc.java` 加入 `multiply`, 再跑測試確認綠燈、截圖、commit `feat`
這裡的feat該怎麼寫

Claude 回應:

`feat:` 後面接一句簡短的英文, 說明這次程式多了什麼功能。照你 subtract 的寫法就好:

```
git add src/main
git commit -m "feat: add multiply"
```

commit 訊息的慣例:

前綴	用在什麼時候	例子
`test:`	只新增或修改測試(紅燈)	`test: multiply returns product (3,5)`
`feat:`	新增功能, 讓測試通過(綠燈)	`feat: add multiply`

前綴	用在什麼時候	例子
`refactor:`	行為不變，只整理程式或測試	`refactor: split negative subtract case into its own test`
`fix:`	修正錯誤	`fix: ...`

幾個小原則：

- 冒號後面空一格，用小寫開頭。
- 用現在式動詞開頭，例如 add、handle、split，不要寫 added 或 successed。
- 簡短就好，細節寫在報告的敘述裡。

commit 前記得先確認綠燈，也就是 `Tests run: 4, Failures: 0, Errors: 0` 和 `BUILD SUCCESS`（如果 subtract 的負數測試有拆開的話，總數是 4）。

第 27 則

學生 **prompt**：

```
@Test public void testDivide()
   {
      assertTrue ("Calc product incorrect", 3 == Calc.divide ( 6, 2));
   }
```

Claude 回應：

這個測試有兩個問題，第二個比較重要。

1. 訊息又是複製過來的

`"Calc product incorrect"` 的 product 是乘積，除法的結果叫 **quotient**（商）。

2. `6 ÷ 2` 分不出 **int** 和 **double**

這是除法最關鍵的一點。不管你的 `divide` 回傳 int 還是 double，`6 ÷ 2` 都會得到 3，連 `3 == 3.0` 在 Java 裡也成立。所以這個測試沒有替你做出任何決定，等於還沒回答老師點名的那個問題：

> whether the division method returns an integer or a floating point number

建議的順序：

1. 先做決定：`divide(7, 2)` 要得到 `3` 還是 `3.5`？想一下理由，例如「Calc 其他方法都是 int，所以保持一致」，或「除法本來就可能有小數，回傳 double 比較正確」。
2. 選一組會讓兩種結果不同的數字：被除數除不盡就可以，像 `7 ÷ 2`。
3. 把你的決定寫成預期值：
   - 選 int：預期 `3`，可以用 `assertTrue(... 3 == ...)` 或 `assertEquals(3, ...)`。

  - 選 double：預期 `3.5`，建議用 `assertEquals(3.5, Calc.divide(7, 2), 0.0001)`。最後那個數字是允許的誤差，比較浮點數時一定要給。

`6 ÷ 2` 不是不能用，可以留著當「整除」的補充測試。但第 1 輪必須是能把型別決定寫死的那個測試。

你打算選 int 還是 double？告訴我你的理由，我幫你檢查測試有沒有真的把它寫死。

第 28 則

學生 **prompt**：

我想選int 並且程式碼如下

```
@Test public void testDivide()
   {
      assertTrue ("Calc quotient incorrect", 3 == Calc.divide ( 7, 2));
   }
```

Claude 回應：

這個測試可以，它確實把你的決定寫死了。

- 如果有人把 `divide` 改成回傳 double，結果會是 `3.5`，`3 == 3.5` 不成立，測試就會失敗。
- `7 ÷ 2` 除不盡，int 和 double 的結果不同，這正是分辨的關鍵。
- 訊息也改成 quotient 了。

進階的小細節（可選）：現在的測試鎖住的是「結果是 3」。如果有人寫成回傳 double 的 `3.0`，測試仍然會通過。想連「回傳型別是 int」一起鎖住，可以先用 int 變數接住結果再比較：

```
int result = Calc.divide(7, 2);
```

如果 `divide` 回傳 double，這一行會編譯失敗，型別也就被固定了。要不要這樣做由你決定，在敘述裡說明即可。

接下來：

1. `mvn clean test`，確認紅燈並截圖。
2. `git add src/test`，然後 `git commit -m "test: divide returns integer quotient (7,2)"`。
3. 實作，確認綠燈，commit `feat`。

這輪做完後，除法還有兩個決定要用測試寫死：

- 除以零：用 `@Test(expected = ...)` 的寫法。你選了 int，想想 Java 的 int 除以零會發生什麼。
- 負數：`divide(-7, 2)` 你預期是多少？先自己想，再寫測試驗證。

別忘了在敘述裡寫下選 **int** 的理由，例如跟 Calc 其他方法的型別保持一致。這是 Q1 評分最看重的部分。

第 29 則

學生 **prompt**:

```
@Test public void testDivide()
   {
      assertTrue ("Calc quotient incorrect", int result = Calc.divide(7,
2););
   }如果如上這樣寫 要在哪裡比較
```

Claude 回應:

這樣寫會編譯錯誤。在 Java 裡，變數宣告（`int result = ...`）必須自成一行，不能放在方法的參數裡。

正確的做法是拆成兩步：先宣告變數接住結果，再拿它來比較。

```
@Test public void testDivide()
   {
      int result = Calc.divide(7, 2);                          // 第 1 步：
用 int 接住結果
      assertTrue ("Calc quotient incorrect", 3 == result);      // 第 2 步：
比較
   }
```

這兩行各自負責一件事：

行	作用	如果 `divide` 寫錯會怎樣
`int result = ...`	鎖住型別：必須回傳 int	回傳 double 會編譯失敗
`assertTrue(... 3 == result)`	鎖住值：必須是 3	回傳 4 之類的錯誤值會測試失敗

另外，你原本那行最後多了一個 `;`（`...(7, 2););`），拆開後就不會有這個問題。

改好後一樣先跑 `mvn clean test`，確認看到 `cannot find symbol: method divide(int,int)` 的紅燈，再 commit。

第 30 則

學生 **prompt**:

```
[ERROR] BUILD FAILURE
[INFO]
------------------------------------------------------------------------
[INFO] Total time:  5.925 s
[INFO] Finished at: 2026-10-22T17:04:39+08:00
[INFO]
------------------------------------------------------------------------
```

```
[ERROR] Failed to execute goal
org.apache.maven.plugins:maven-compiler-plugin:3.16.0:testCompile
(default-testCompile) on project calc-tdd: Compilation failure
[ERROR] /Users/ning/Documents/交大/軟體測試
/calc-tdd/src/test/java/CalcTest.java:[28,24] cannot find symbol
[ERROR]   symbol:   method divide(int,int)
[ERROR]   location: class Calc
[ERROR] -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e
switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please
read the following articles:
[ERROR] [Help 1]
http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException
```

Claude 回應:

紅燈正確。第 28 行找不到 `divide(int,int)`, 因為這個方法還不存在。這就是除法第 1 輪的紅燈證據, 請截圖保存。

接下來:

1. commit 測試:

```
git add src/test
git commit -m "test: divide returns integer quotient (7,2)"
```

2. 到 **`Calc.java`** 加入 **`divide`**, 寫法參考你的 **multiply**。寫之前想一下:Java 裡兩個 int 相除, 會自動得到整數結果還是小數？這決定了你的實作要不要做額外的處理。

3. 確認綠燈:`mvn clean test` 應該顯示 `Failures: 0, Errors: 0` 和 `BUILD SUCCESS`。截圖後 commit:

```
git add src/main
git commit -m "feat: add divide (integer division)"
```

之後還有兩個除法的決定要寫成測試, 每個都是新的一輪:

決定	提示	想想看
除以零	`@Test(expected = 例外類別.class)`, 方法裡呼叫 `Calc.divide(某數, 0)`	Java 的 int 除以零會拋哪一種例外？寫好後會是紅燈還是直接綠燈？
負數	例如 `divide(-7, 2)`	Java 的整數除法往哪個方向取整？預期值是 -3 還是 -4？

這兩個測試很可能一寫就通過，因為 Java 本身的行為已經符合預期。這沒關係，跟 subtract 的負數測試一樣，在敘述裡說明「為什麼沒有紅燈」就好。

第 31 則

學生 **prompt**：

```
@Test(expected = 例外類別.class)這是要寫在哪的

@Test public void testDivideByZero()
   {
      int result = Calc.divide(7, 0);
      assertTrue ("Calc quotient incorrect", 3 == result);
   }
}
```

Claude 回應：

`@Test(expected = ...)` 要取代方法上面原本的 `@Test`。它告訴 JUnit：「這個測試應該拋出這種例外，有拋才算通過。」

```
@Test(expected = 例外類別.class)
public void testDivideByZero()
   {
      Calc.divide(7, 0);
   }
```

跟你的版本比，改了三個地方：

改了什麼	為什麼
`@Test` 改成 `@Test(expected = ...)`	讓 JUnit 知道「拋出例外」才是正確結果
拿掉 `assertTrue` 那行	除以零根本不會有結果，程式在 `divide` 那行就會拋例外，後面的比較永遠不會執行。`3 == result` 也沒有意義，7 除以 0 不會等於 3。
拿掉 `int result =`	不需要接住結果，只要呼叫就好

例外類別 要換成什麼，是你要做的決定：

- 做法 **A**：沿用 **Java** 內建的行為。Java 的 int 除以零會自動拋出 **`ArithmeticException`**。如果你接受這個行為，就寫 `ArithmeticException.class`。這個測試很可能一寫就通過，因為你的 `divide` 已經會自動拋出它。
- 做法 **B**：自己定義行為。例如你覺得除以零是「參數不合法」，想拋出 `IllegalArgumentException` 並附上清楚的訊息。這時測試會是紅燈，因為程式目前拋的是另一種例外，你需要修改 `divide` 讓它通過。

兩種都合理。選好之後在敘述裡寫下理由。做法 B 會多一個完整的紅綠循環，可以寫的內容比較多。

另外：你貼的程式碼最後有一個 }。如果那是整個 class 的結尾就沒問題，只要確認新的測試方法在它前面。

第 32 則

學生 **prompt**：

```
divide by zero 的feat要寫什麼
```

Claude 回應：

要看你選了哪一種做法。

做法 **A**：沿用 **`ArithmeticException`**

不需要 **feat commit**。這個測試一寫就會通過，你沒有修改任何程式，跟 subtract 的負數測試一樣，只要一個 test commit：

```
git add src/test
git commit -m "test: divide by zero throws ArithmeticException (passes
immediately)"
```

敘述裡說明：「Java 的整數除以零本來就會拋出 ArithmeticException，所以不需要修改 `divide`。」

做法 **B**：改成 **`IllegalArgumentException`**

這會是完整的紅綠循環，有兩個 commit：

```
# 紅燈：測試期待 IllegalArgumentException，但程式拋的是 ArithmeticException
git add src/test
git commit -m "test: divide by zero throws IllegalArgumentException"

# 綠燈：修改 divide 讓它通過之後
git add src/main
git commit -m "feat: divide throws IllegalArgumentException on zero divisor"
```

怎麼確認自己屬於哪一種？看寫完測試後，改程式之前跑 `mvn clean test` 的結果：

- 直接 `BUILD SUCCESS`：做法 A，只要 test commit。
- 出現 `Unexpected exception, expected<...IllegalArgumentException> but was<...ArithmeticException>`：做法 B，這就是你的紅燈，要截圖。

第 33 則

學生 **prompt**：

```
ning@xujianingdeMacBook-Air calc-tdd % git log --oneline
287265e (HEAD -> main) test: divide by negetive.
846a8fa test: divide by zero throws ArithmeticException (passes immediately)
1b761b5 feat: add divide (integer division)
1b8e430 test: divide returns integer quotient (7,2)
```

```
75093c5 feat: add multiply
c72087c test: multiply returns product (3,5)
4b6acb3 refactor: split negative subtract case into its own test
f60184a test: subtract handles negative result (passes immediately)
dd5927b feat: add subtract
a5c68be test: subtract returns difference (5,3)
ae313b4 chore: starter Calc (add only) + Maven/CI setup
```

Claude 回應:

git 紀錄很完整, Q1 的實作部分基本上完成了。整理一下你的 TDD 歷程:

運算	紀錄	狀態
subtract	紅 → 綠 → 補充負數測試 → 重構	✓ 完整, 還有一次重構
multiply	紅 → 綠	✓
divide	紅 → 綠(int) → 除以零 → 負數	✓ 三個決定都用測試寫死了

要修的一個小地方:最新的 commit 有錯字, `negetive` 應該是 negative, 訊息也看不出這個測試定義了什麼。改成能說明你決定的版本, 例如:

```
git commit --amend -m "test: divide truncates toward zero for negative operands"
```

如果這個測試一寫就通過, 記得在訊息加上 `(passes immediately)`, 跟其他補充測試保持一致。

Q1 剩下的事:

1. 最終綠燈截圖(必交):執行 `mvn clean verify`, 截圖要看得到全部的 `Tests run` 和 `BUILD SUCCESS`。用 `verify` 會順便跑覆蓋率和靜態分析, 選做 Q3 用得到。如果出現覆蓋率不足的錯誤, 貼給我看。
2. 複製 **`git log --oneline`** 的輸出, 放進報告附錄 C。
3. 寫敘述:照 git log 一輪一輪寫, 每一輪寫測試、紅燈、讓它變綠的修改、重構。你有幾個真實插曲很值得寫:
   - subtract 第一次寫錯預期值(寫成 5)。
   - 原本想用 `3×1`, 後來發現 `return a` 也能通過, 改成 `3×5`。
   - 除法選 int 的理由, 以及用 `int result =` 鎖住回傳型別。
   - 為什麼有些測試一寫就通過。
4. 選做, 可以多一次重構:其他測試都用 `assertTrue(... == ...)`。如果統一改成 `assertEquals`, 失敗訊息會更清楚。改完跑測試確認綠燈, 再 commit `refactor:`。

敘述寫完可以貼給我, 我幫你檢查有沒有漏掉評分會看的項目, 但不改寫你的文字。

Q1 結束後就能開始 **Q6**, 由我當 agent 加入 modulo 和 power。你之前已經決定了負數取餘數、`power(0,0)` 和負指數的行為, 開始前我會再問你取餘數除以零和 `power(0, -1)` 這兩個決定。

第 34 則

學生 **prompt**:

〔學生貼上 `mvn clean verify` 的完整輸出，重點節錄如下〕

```
ning@xujianingdeMacBook-Air calc-tdd % mvn clean verify
...
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
...
[INFO] --- jacoco:0.8.12:check (check) @ calc-tdd ---
[INFO] Loading execution data file /Users/ning/Documents/交大/軟體測試
/calc-tdd/target/jacoco.exec
[INFO] Analyzed bundle 'calc-tdd' with 1 classes
[WARNING] Rule violated for bundle calc-tdd: lines covered ratio is 0.80, but
expected minimum is 1.00
[INFO]
------------------------------------------------------------------------
[ERROR] BUILD FAILURE
...
[ERROR] Failed to execute goal org.jacoco:jacoco-maven-plugin:0.8.12:check
(check) on project calc-tdd: Coverage checks have not been met. See log for
details. -> [Help 1]
```

Claude 回應:

7 個測試全部通過，你的 Q1 沒有問題。這次失敗是我設定的覆蓋率規則太嚴格，不是你的程式有錯。

原因:Java 的 class 如果沒有寫建構子，會自動產生一個看不見的 `public Calc()`。你的測試全部是呼叫 `Calc.add(...)` 這種 static 方法，從來沒有 `new Calc()`，所以那個隱藏的建構子一直沒被執行到。

JaCoCo 算到 5 行，`public class Calc` 那行加上四個方法的 return，其中 4 行有執行到:4 ÷ 5 = **0.80**，剛好就是錯誤訊息裡的數字。我設定的門檻是 100%，所以失敗了。

自己確認:打開 `target/site/jacoco/index.html`，點進 `Calc`，會看到 `public class Calc` 那一行被標成紅色，其他行都是綠色。

修正方式:打開 `pom.xml`，找到下面這段，把整段刪掉，只保留上面的 BRANCH 規則:

```
<limit>
  <counter>LINE</counter>
  <value>COVEREDRATIO</value>
  <minimum>1.00</minimum>
</limit>
```

之後執行 `mvn clean verify`，應該就會 `BUILD SUCCESS`。commit 時標明這是設定調整，不是 Q1 的 TDD 步驟:

```
git add pom.xml
git commit -m "chore: drop line-coverage rule (implicit constructor is never
called)"
```

另一種做法(選做):在 `Calc` 裡加一個 `private Calc() {}`。這是 Java 常見的「只有 static 方法的工具類別」寫法,讓別人無法 `new Calc()`。不過 private 建構子一樣不會被執行到,JaCoCo 會不會把它從統計中排除,要你實際跑過才知道。所以先刪 LINE 規則最簡單。

Q1 要交的「全部測試通過」截圖,用 `mvn clean test` 的畫面就可以:`Tests run: 7, Failures: 0, Errors: 0` 加上 `BUILD SUCCESS`。`verify` 的覆蓋率檢查是給選做的 Q3 用的。這次的覆蓋率問題也可以寫進 Q3:覆蓋率工具抓到了一行從沒被執行的程式碼,你判斷它不影響行為,所以調整了規則。

第 35 則

學生 **prompt**:

〔學生貼上 `mvn clean verify` 的完整輸出,重點節錄如下〕

```
ning@xujianingdeMacBook-Air calc-tdd % mvn clean verify
...
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
...
[INFO] All coverage checks have been met.
...
[INFO] --- spotbugs:4.8.6.6:spotbugs (spotbugs) @ calc-tdd ---
     [java] The following errors occurred during analysis:
     [java]   Error scanning java/lang/Object for referenced classes
     [java]     java.lang.IllegalArgumentException: Unsupported class file
major version 71
...
[ERROR] BUILD FAILURE
...
[ERROR] Failed to execute goal
com.github.spotbugs:spotbugs-maven-plugin:4.8.6.6:spotbugs (spotbugs) on
project calc-tdd: Execution spotbugs of goal
com.github.spotbugs:spotbugs-maven-plugin:4.8.6.6:spotbugs failed: Java
returned: 1 -> [Help 1]
```

Claude 回應:

覆蓋率這次過了(`All coverage checks have been met`),7 個測試也全部通過。這次失敗的是靜態分析工具 **SpotBugs**,跟你的程式一樣沒有關係。

原因:`Unsupported class file major version 71` 代表你電腦上的 Java 版本比 **SpotBugs** 支援的還新。Homebrew 安裝 Maven 時裝了最新版的 Java,而我在 pom 裡指定的 SpotBugs 4.8.6 讀不懂這麼新的 Java 檔案格式。可以用 `java -version` 確認你的版本。

建議做法:本機跳過 **SpotBugs**,交給 **CI** 跑

```
mvn clean verify -Dspotbugs.skip=true
```

這樣測試和覆蓋率照常檢查，只略過 SpotBugs。GitHub Actions 的 CI 設定用的是 **Java 17**，SpotBugs 在那裡可以正常運作。所以 Q3 的靜態分析結果，之後看 CI 的紀錄和截圖就好。

如果想在本機也跑 **SpotBugs**（選做）：另外安裝一個 Java 21，執行時改用它：

```
brew install openjdk@21
JAVA_HOME=/opt/homebrew/opt/openjdk@21 mvn clean verify
```

別忘了把 **pom** 的修改 **commit** 起來：

```
git add pom.xml
git commit -m "chore: drop line-coverage rule (implicit constructor is never
called)"
```

到這裡 **Q1** 的程式部分已經完成：7 個測試全過，git 紀錄完整。剩下截最終綠燈的圖（`mvn clean test` 就夠）和寫敘述。寫好後可以貼給我，我檢查有沒有漏項。如果想先往下做，告訴我，我們就開始 Q6。

第 36 則

學生 **prompt**：

〔學生貼上 `mvn clean verify -Dspotbugs.skip=true` 的完整輸出，重點節錄如下〕

```
ning@xujianingdeMacBook-Air calc-tdd % mvn clean verify -Dspotbugs.skip=true
...
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
...
[INFO] All coverage checks have been met.
...
[INFO] Spotbugs plugin skipped
[INFO]
------------------------------------------------------------------------
[INFO] BUILD SUCCESS
```

Claude 回應：

`BUILD SUCCESS`，三項檢查都通過：

檢查	結果
測試	7 個全過
覆蓋率（JaCoCo）	`All coverage checks have been met`
靜態分析（SpotBugs）	本機跳過，之後在 CI 上跑

這張畫面可以當作 Q1「全部測試通過」的截圖，順便也是 Q3 覆蓋率的證據。

Q1 收尾：

1. 把 pom 的修改 commit 起來（如果還沒做）：

```
git add pom.xml
git commit -m "chore: drop line-coverage rule (implicit constructor is
never called)"
```

2. 執行 `git log --oneline`，把輸出複製到報告附錄 C。
3. 照報告模板 1.x 節，一輪一輪寫敘述。寫好貼給我，我幫你檢查有沒有漏項。

Q6 準備開始時告訴我。我會照規則進行：每輪只寫一個失敗的測試，附上紅燈輸出，然後停下來等你審查並 commit，你說可以了我才寫實作。開始前還有幾個決定要你來做：

- `modulo(7, 0)` 要怎麼處理？
- 你選了負指數用整數截斷，那 `power(0, -1)`（等於 1/0）要怎麼處理？
- `power(-1, -3)` 截斷後是 -1 還是 0？
- 結果超過 int 範圍（溢位）時怎麼處理？

這些決定我開始 Q6 時會正式問一次，讓它們留在 AI 使用紀錄裡。

第 37 則（Q6 開始）

學生 **prompt**：

〔附圖：Q1 各輪紅燈、綠燈截圖，共 10 張〕

```
ning@xujianingdeMacBook-Air calc-tdd % git log --oneline
047a85a (HEAD -> main) chore: drop line-coverage rule (implicit constructor
is never called)
287265e test: divide by negetive.
846a8fa test: divide by zero throws ArithmeticException (passes immediately)
1b761b5 feat: add divide (integer division)
1b8e430 test: divide returns integer quotient (7,2)
75093c5 feat: add multiply
c72087c test: multiply returns product (3,5)
4b6acb3 refactor: split negative subtract case into its own test
f60184a test: subtract handles negative result (passes immediately)
dd5927b feat: add subtract
a5c68be test: subtract returns difference (5,3)
ae313b4 chore: starter Calc (add only) + Maven/CI setup
以上是log內容 並且所有相關截圖 幫我和q2～q6全部做完然後整理成一份詳細精準的作業
```

Claude 回應（訊息 **1**）：

Q1 的實作部分完成了：7 個測試全部通過，git 紀錄也完整。接下來各部分的分工如下。

我可以做完的：

- **Q6**（必做）：由我當 agent，一輪一輪照規則做。我會直接修改你電腦上 `calc-tdd` 裡的檔案，每輪只寫一個失敗的測試，然後停下來。你跑 `mvn test` 確認紅燈、審查測試、commit 之後，我才寫實作。
- 最後的完整報告：把 Q1 和 Q6 的測試、最終版 Calc、git log、截圖清單、Q6 每輪的敘述、違規紀錄、決定紀錄、AI 使用紀錄整理成一份。

必須由你做的：

- **Q1** 的敘述：題目規定不能用 AI。報告裡我會放好你的 git log、測試程式碼和截圖順序，但每一輪的文字要你自己寫。
- 按 **commit**：Q6 規則 (b) 要求由你審查並 commit 每個測試，git 歷史必須看得出這個順序。
- **Q2**、**Q3** 的 **GitHub** 操作：push 到 GitHub、在 Actions 頁面截圖要用你的帳號。我會給你逐步指令，Q2 後半「讓 agent 修壞掉的 build」由我來當那個 agent。
- **AI** 使用紀錄：作業要求逐字的 prompt 和完整回應，最後要從這個對話直接複製。我沒辦法替你匯出。

先回答下面幾個問題，Q6 規則 (c) 規定這些決定要在寫程式之前用測試定下來。

〔選擇題〕

問題	選項	學生選擇
Q6(c)：modulo 除以零，例如 modulo(7, 0)，要怎麼處理？	拋 ArithmeticException (Recommended)／拋 IllegalArgumentException	拋 **ArithmeticException (Recommended)**
Q6(c)：你選了負指數用整數截斷，那 power(0, -1)（等於 1/0）要怎麼處理？	拋 ArithmeticException (Recommended)／回傳 0	拋 **ArithmeticException (Recommended)**
power 的結果超過 int 範圍時（例如 power(2, 31)）怎麼處理？	拋 ArithmeticException (Recommended)／不處理，讓它自然溢位／不寫進規格	拋 **ArithmeticException (Recommended)**
選做題要做哪些？Q4 和 Q5 需要你自己挑一個大型的既有專案，並先自己完成一次。（可複選）	Q2 CI＋弄壞 build／Q3 覆蓋率／靜態分析／Q7 性質測試／只做必做題	**Q2 CI**＋弄壞 **build**、**Q3** 覆蓋率／靜態分析、**Q7** 性質測試

〔Agent 動作〕請求存取學生電腦的 calc-tdd 資料夾（學生核准）；讀取 Calc.java、CalcTest.java、git 狀態；在 CalcTest.java 末尾新增 `testModuloByZero`（未修改 Calc.java）；在沙盒編譯並執行測試取得紅燈輸出。

Claude 回應（訊息 **2**）：

Q6 第 **1** 輪：紅燈。測試已寫好，實作還沒開始。

我在你的 `CalcTest.java` 最後加了一個測試，沒有動 **`Calc.java`**：

```
   // Q6 cycle 1 (agent-written): decision (c) - modulo by zero throws
ArithmeticException
   @Test(expected = ArithmeticException.class)
```

```java
public void testModuloByZero()
{
    Calc.modulo(7, 0);
}
```

我自己跑出來的失敗輸出：

```
CalcTest.java:48: error: cannot find symbol
      Calc.modulo(7, 0);
  symbol:   method modulo(int,int)
  location: class Calc
```

為什麼第一個測試是「除以零」而不是一般的 `7 % 3`：規則 (c) 要求先用測試定下決定，再寫程式碼。Java 的 `%` 天生就會在除以零時拋例外，也天生讓結果的正負號跟著被除數。如果第一輪就寫出 `a % b`，後面的決定測試會一加進來就通過，等於實作跑在規格前面。所以我的順序是：先測除以零，下一輪再測負數運算元，每個決定都先有失敗的測試，才有讓它通過的程式碼。

請你：

1. 審查這個測試。報告的「審查意見」要寫你的看法，例如它是否符合你選的「拋 ArithmeticException」。
2. 執行 `mvn clean test`，確認紅燈並截圖，這就是規則 (a) 要的失敗輸出證據。
3. commit 測試：

   ```
   git add src/test
   git commit -m "test(q6): modulo by zero throws ArithmeticException"
   ```

4. 回覆「OK」或貼上 commit 結果，我才會寫實作。如果要改測試，直接告訴我改哪裡。

第 38 則

學生 **prompt**：

〔附圖：Q6 第 1 輪紅燈畫面〕

1. 他好像沒有說是ArithmeticException 因為此處還沒在calc.java加modulo的函式
2. 如上圖
3. 這樣我還要執行這兩行嗎

Claude 回應：

1. 你的觀察是對的，很適合直接寫進審查意見。

目前的紅燈是編譯失敗，`modulo` 根本不存在，所以 JUnit 還沒機會檢查「有沒有拋 ArithmeticException」。換句話說，這個紅燈只證明了「功能還不存在」，還沒證明「除以零的規格」會被檢查到。

規格要等實作出現後才會真正生效：

- 如果實作拋了 ArithmeticException，測試通過。
- 如果實作沒拋，例如回傳 0，JUnit 會顯示 `Expected exception: java.lang.ArithmeticException`，測試失敗。

下一步實作完成後，我會額外示範一次：先用一個不拋例外的錯誤版本跑測試，證明它真的會抓到。這樣你的觀察就有實際驗證，報告也更完整。

審查意見可以這樣寫（用你自己的話）：「紅燈來自編譯錯誤，尚未驗證例外型別；要求 agent 實作後證明測試能抓到不拋例外的錯誤實作。」

2. 截圖是合格的紅燈證據。錯誤位置第 48 行，`cannot find symbol: method modulo(int,int)`，跟我跑出來的一致。

3. 要，這兩行一定要執行。規則 (b) 要求「你審查並 commit 測試之後，agent 才能實作」，git 歷史要看得出這個順序。

```
git add src/test
git commit -m "test(q6): modulo by zero throws ArithmeticException"
```

commit 完把 `git log --oneline -3` 的結果貼給我，我才開始寫 `modulo`。

第 39 則

學生 **prompt**：

```
2.幫我翻譯成英文 紅燈來自編譯錯誤，還未驗證例外型別
ning@Mac calc-tdd % git log --oneline -3
c6afa1a (HEAD -> main) test(q6): modulo by zero throws ArithmeticException
047a85a chore: drop line-coverage rule (implicit constructor is never called)
287265e test: divide by negetive.
```

Claude 回應（訊息 **1**，執行動作前）：

審查意見的英文：

> The red bar comes from a compilation error, so the exception type has not been verified yet.

想再補一句要求的話：

> I asked the agent to show, after implementing, that the test fails on an incorrect implementation that does not throw.

commit 紀錄正確：test commit `c6afa1a` 在實作之前。現在開始第 1 輪的綠燈。

〔Agent 動作〕在沙盒中暫時把 modulo 寫成錯誤版本 `return 0;` 執行測試，得到 `AssertionError: Expected exception: java.lang.ArithmeticException`（Tests run: 8, Failures: 1），證明測試能抓到不拋例外的實作，之後還原；在學生的 Calc.java 加入 fake 實作（永遠拋 ArithmeticException），未修改測試檔；沙盒執行 OK (8 tests)。

Claude 回應（訊息 **2**）：

Q6 第 1 輪：綠燈。

我只改了 `Calc.java`，完全沒碰測試檔。加入的程式碼：

```
   // Q6 cycle 1 (agent-written): minimal code to pass testModuloByZero.
   // Deliberately "fake it": the next test (negative operands) will force
the real formula.
   static public int modulo (int a, int b)
   {
      throw new ArithmeticException("modulo by zero");
   }
```

我這邊的結果是 `OK (8 tests)`。

為什麼故意「永遠拋例外」？這是 TDD 的 fake it 技巧：只寫剛好能通過目前測試的程式碼。真正的 `a % b` 留到下一輪，由「負數運算元」那個決定測試來逼出來。這樣規則 (c) 的每個決定，都是先有失敗的測試才有實作。

這一輪我自己做了兩個沒問你的小決定，會記進報告的 **2.4** 節：

1. 用 fake it 策略，而不是直接寫 `a % b`。
2. 例外訊息寫成 `"modulo by zero"`。測試沒有檢查訊息內容，所以不影響行為。

請你：

1. 執行 `mvn clean test`，確認 `Tests run: 8, Failures: 0, Errors: 0` 並截圖。
2. 用 `git diff --stat` 確認只有 `src/main/java/Calc.java` 有變動，這是證明 agent 沒碰測試的證據。
3. commit 實作：

   ```
   git add src/main
   git commit -m "feat(q6): add modulo (minimal: throw on any input)"
   ```

4. 回覆後，我開始第 2 輪：負數運算元，`modulo(-7,3) == -1`、`modulo(7,-3) == 1`、`modulo(-7,-3) == -1`。

第 40 則

學生 **prompt**：

〔附圖：Q6 第 1 輪綠燈畫面〕

〔Agent 動作〕檢查學生 repo 的 git log 與狀態，發現 Calc.java 仍未 commit。

Claude 回應：

綠燈確認：`Tests run: 8, Failures: 0, Errors: 0`, `BUILD SUCCESS`。這張是第 1 輪的綠燈證據。

不過我看了一下你的 repo，實作還沒 **commit**：`Calc.java` 仍是 modified 狀態，最新的 commit 還是 test 那個。

先 commit，第 1 輪才算完成：

```
git add src/main
git commit -m "feat(q6): add modulo (minimal: throw on any input)"
```

我必須等這個 commit 完成才能寫第 2 輪的測試，不然測試和實作的修改會混在一起，git 歷史就看不出順序。commit 完回覆一聲就好，我會自己檢查 git log 再開始。

第 41 則

學生 **prompt**：

好了

〔Agent 動作〕確認 git log 最新為 `340b317 feat(q6): add modulo (minimal: throw on any input)` 且工作區乾淨；在 CalcTest.java 新增 `testModuloNegativeOperands`（未修改 Calc.java）；沙盒執行取得紅燈。

Claude 回應：

實作 commit `340b317` 已確認，第 1 輪完成。

Q6 第 **2** 輪：紅燈。測試已寫好，實作還沒開始。

我只在 `CalcTest.java` 加了一個測試，沒有動 `Calc.java`：

```
   // Q6 cycle 2 (agent-written): decision (c) - modulo with negative
operands.
   // The result takes the sign of the dividend (truncated division, same as
Java's %).
   @Test public void testModuloNegativeOperands()
   {
      assertTrue ("Calc remainder incorrect for (-7, 3)", -1 == Calc.modulo
(-7, 3));
      assertTrue ("Calc remainder incorrect for (7, -3)",  1 == Calc.modulo
(7, -3));
      assertTrue ("Calc remainder incorrect for (-7, -3)", -1 == Calc.modulo
(-7, -3));
   }
```

我跑出來的失敗輸出：

```
1) testModuloNegativeOperands(CalcTest)
java.lang.ArithmeticException: modulo by zero
    at Calc.modulo(Calc.java:31)
Tests run: 9,  Failures: 1
```

這次是行為上的紅燈，跟第 1 輪的編譯錯誤不同：`modulo` 已經存在，但上一輪的 fake 實作永遠拋例外，所以這個測試抓到了它。在 Maven 裡應該會顯示成 `Errors: 1`，因為測試是被例外中斷，而不是斷言失敗。

這個測試定義了你選的規則「正負號跟被除數」：

輸入	預期	為什麼
`(-7, 3)`	-1	被除數負 → 結果負
`(7, -3)`	1	被除數正 → 結果正，即使除數是負的
`(-7, -3)`	-1	兩個都負 → 跟著被除數，是負的

如果選的是「跟除數」（Python 的規則），`(-7,3)` 會是 2、`(7,-3)` 會是 -2，所以這三組可以分辨兩種規則。

我做的兩個小決定，會記進報告：一個測試方法裡放三個斷言，因為三組共同定義同一個決定；斷言風格沿用你 Q1 的 `assertTrue`。

請你：

1. 審查測試。
2. 執行 `mvn clean test`，確認紅燈並截圖。
3. commit：

   ```
   git add src/test
   git commit -m "test(q6): modulo result takes the sign of the dividend"
   ```

4. 回覆「好了」，我會檢查 git log 後再實作。

第 42 則

學生 **prompt**：

〔附圖：Q6 第 2 輪紅燈畫面〕

好了

〔Agent 動作〕確認 git log 最新為 `a170f27 test(q6): modulo result takes the sign of the dividend`；把 Calc.java 的 modulo 改為 `return a % b;`（未修改測試檔）；沙盒執行 OK (9 tests)。

Claude 回應：

測試 commit `a170f27` 已確認，順序正確。

關於你的紅燈截圖：畫面只拍到 `BUILD FAILURE`，最關鍵的那幾行被捲上去了。往上捲一點，把下面這段也截進去，證據會更完整：

```
Tests run: 9, Failures: 0, Errors: 1
testModuloNegativeOperands ... ArithmeticException: modulo by zero
```

Q6 第 **2** 輪：綠燈。

只改了 `Calc.java`，把上一輪的 fake 換成真正的公式：

```
   static public int modulo (int a, int b)
   {
      return a % b;
   }
```

我這邊的結果是 `OK (9 tests)`。

為什麼一行 **`a % b`** 就同時滿足兩個決定：

- Java 的 `%` 是截斷除法的餘數，結果的正負號跟著被除數，正好符合你選的規則。
- `b == 0` 時 Java 會自動拋 `ArithmeticException`，所以第 1 輪的測試仍然通過。

副作用：例外訊息從我寫的 `"modulo by zero"` 變成 Java 內建的 `"/ by zero"`。測試只檢查例外類型，不檢查訊息，所以仍是綠燈。報告裡我會把這點記成「測試沒有規範訊息內容」。

請你：

1. 執行 `mvn clean test`，確認 `Tests run: 9, Failures: 0, Errors: 0`，截圖。
2. commit：

   ```
   git add src/main
   git commit -m "feat(q6): modulo uses Java % (sign follows dividend)"
   ```

3. 回覆「好了」後，開始做 **power**，第 3 輪是 `power(0, 0) == 1`。

第 43 則

學生 **prompt**：

〔附圖：第 2 輪補拍的紅燈畫面、第 2 輪綠燈畫面〕

好了

〔Agent 動作〕確認 git log 最新為 `743b568 feat(q6): modulo uses Java % (sign follows dividend)`；在 CalcTest.java 新增 `testPowerZeroToZero`（未修改 Calc.java）；沙盒執行取得紅燈。

Claude 回應：

modulo 的兩輪都完成了：實作 commit 是 `743b568`。你補的截圖也拍到了 `testModuloNegativeOperands:55 » Arithmetic modulo by zero` 和 `Errors: 1`，紅燈證據完整了。

Q6 第 **3** 輪：紅燈（開始做 **power**）。測試已寫好，實作還沒開始。

我只在 `CalcTest.java` 加了一個測試：

```
   // Q6 cycle 3 (agent-written): decision (c) - power(0, 0) returns 1
   @Test public void testPowerZeroToZero()
   {
      assertTrue ("Calc power incorrect for (0, 0)", 1 == Calc.power (0, 0));
   }
```

我跑出來的失敗輸出：

```
CalcTest.java:63: error: cannot find symbol
      ... Calc.power (0, 0));
  symbol:   method power(int,int)
  location: class Calc
```

為什麼 **power** 從 **`power(0,0)`** 開始：理由跟 modulo 一樣。如果第一輪先測 `power(2,3)` 並寫出迴圈，迴圈「乘 0 次」本來就會回傳 1，`power(0,0)` 的決定就會在測試出現前被實作偷偷決定了。先把你的決定寫成測試，後面的實作才是被規格逼出來的。

接下來 **power** 的規劃（每輪仍然只寫一個測試）：

輪	測試	定義的決定
3	`power(0,0) == 1`	(c) 0 的 0 次方
4	`power(2,3) == 8`	一般情況
5	`power(2,-1) == 0`、`power(1,-3) == 1`、`power(-1,-3) == -1`	(c) 負指數：整數截斷
6	`power(0,-1)` 拋 ArithmeticException	你選的 0 的負次方
7	`power(2,31)` 拋 ArithmeticException	你選的溢位處理

請你：

1. 審查測試。
2. 執行 `mvn clean test`，確認紅燈並截圖。
3. commit：

   ```
   git add src/test
   git commit -m "test(q6): power(0,0) returns 1"
   ```

4. 回覆「好了」。

第 44 則

學生 **prompt**：

〔附圖：Q6 第 3 輪紅燈畫面〕

好了 之後作業q6的這些過程麻煩你直接在我電腦上跑好 截好圖存好

〔Agent 動作〕確認 git log 最新為 `0578a17 test(q6): power(0,0) returns 1`;檢查學生電腦的 Linux 環境(只有 Java 11 執行環境, 沒有 Maven 和 javac);在 Calc.java 加入 fake 實作 `return 1;`(未修改測試檔);沙盒執行 OK (10 tests);把 Agent 端測試紀錄寫入學生 repo 的 `q6-evidence/agent-run-log.txt`。

Claude 回應:

Q6 第 **3** 輪:綠燈。我在 `Calc.java` 加了最小實作(fake it), 我這邊跑出 `OK (10 tests)`:

```
static public int power (int base, int exponent)
{
   return 1;
}
```

下一輪的 `power(2,3) == 8` 會逼出真正的迴圈。

關於「直接在你電腦上跑好、截好圖」, 有兩件事我做不到:

1. 我沒辦法截你 **Mac** 的螢幕。我能用的是你電腦上一個隔離的 Linux 環境, 只看得到 `calc-tdd` 資料夾, 沒有 Maven 也沒有 Java 編譯器, 碰不到你的 macOS 桌面。所以 `mvn` 只能在你的終端機跑。
2. **commit** 不該由我來做。規則 (b) 寫明要由你審查並 commit 測試, agent 才能實作。如果由我自己寫測試、自己 commit、自己實作, 正好就是報告裡要舉報的違規。

我能幫你省掉大部分手動操作:

- 我每輪在我這邊實際跑出的紅燈和綠燈輸出, 已經存到你的 `calc-tdd/q6-evidence/agent-run-log.txt`, 第 1 到 3 輪都在裡面, 之後每輪會自動追加。
- 你每一步只要複製貼上一行指令, 它會跑 `mvn`、把輸出存成文字檔, 然後 commit。文字紀錄可以取代截圖, 而且比截圖更精確。

現在請貼上這一行(第 **3** 輪綠燈):

```
mvn clean test > q6-evidence/cycle3-green.txt 2>&1 && tail -8
q6-evidence/cycle3-green.txt && git add src/main && git commit -m "feat(q6):
add power (minimal: return 1)"
```

它只會在測試全部通過時才 commit, 不會把壞掉的程式 commit 進去。`git add src/main` 只加程式檔, `q6-evidence/` 裡的紀錄檔不會混進 commit。

之後每一輪你會收到兩行指令:

- 紅燈那步:你看過我寫的測試後(這就是你的審查), 貼上指令, 它會存紅燈輸出並 commit 測試。
- 綠燈那步:貼上指令, 它會存綠燈輸出並 commit 實作。

整份作業只剩最後一張「全部測試通過」的截圖一定要你自己截, 那是必交項目。

這一輪我自己做了兩個沒問你的小決定，會記進報告：參數取名為 `base` 和 `exponent`，以及用 fake it 回傳 1。貼完指令回覆「好了」，我就寫第 4 輪的測試。

第 45 則

學生 **prompt**：

〔附圖：執行第 3 輪綠燈指令的畫面〕

這樣嗎

Claude 回應（訊息 **1**，執行動作前）：

對，完全正確：測試通過才 commit，實作 commit 是 `2c47f1a`，綠燈紀錄存在 `q6-evidence/cycle3-green.txt`。接著寫第 4 輪。

〔Agent 動作〕確認 git log 最新為 `2c47f1a`；在 CalcTest.java 新增 `testPowerPositiveExponent`（未修改 Calc.java）；沙盒執行取得紅燈；更新 agent-run-log.txt。

Claude 回應（訊息 **2**）：

Q6 第 **4** 輪：紅燈。

我只在 `CalcTest.java` 加了一個測試：

```
// Q6 cycle 4 (agent-written): ordinary positive exponent
@Test public void testPowerPositiveExponent()
{
   assertTrue ("Calc power incorrect for (2, 3)", 8 == Calc.power (2, 3));
}
```

我跑出來的失敗輸出：

```
1) testPowerPositiveExponent(CalcTest)
java.lang.AssertionError: Calc power incorrect for (2, 3)
Tests run: 11,  Failures: 1
```

這次是斷言失敗（**Failure**）：上一輪的 fake 實作 `return 1` 回傳了 1 而不是 8，被新測試抓到了。這就是三角定位：用第二個例子逼出真正的實作。

`(2, 3)` 的選法：底數和指數不同，結果 8 也跟 `2*3=6`、`2+3=5`、`3^2=9` 都不同，所以寫錯成乘法、加法或底數指數對調都會被抓到。

審查完測試後，貼上這一行（存紅燈紀錄並 commit 測試）：

```
mvn clean test > q6-evidence/cycle4-red.txt 2>&1; grep -E "Tests
run:|<<<|incorrect" q6-evidence/cycle4-red.txt | tail -4; git add src/test &&
git commit -m "test(q6): power(2,3) returns 8"
```

這行在 `mvn` 後面用的是 `;` 而不是 `&&`，因為紅燈時 mvn 一定會失敗，但我們仍然要 commit 測試。畫面上應該會看到 `Failures: 1` 和 commit 成功的訊息。

回覆「好了」，我就寫實作。

第 46 則

學生 **prompt**：

〔附圖：第 4 輪紅燈與 test commit 畫面〕

好姥

〔Agent 動作〕確認 git log 最新為 `2052f7b test(q6): power(2,3) returns 8`；把 Calc.java 的 power 改為迴圈連乘（未修改測試檔）；沙盒執行 OK (11 tests)；更新 agent-run-log.txt。

Claude 回應：

第 4 輪的測試 commit `2052f7b` 已確認，紅燈紀錄也存好了。

Q6 第 **4** 輪：綠燈。只改了 `Calc.java`，把 fake 換成迴圈：

```
   // Q6 cycle 4 (agent-written): repeated multiplication.
   // exponent == 0 runs the loop zero times, so power(0, 0) == 1 still
holds.
   static public int power (int base, int exponent)
   {
      int result = 1;
      for (int i = 0; i < exponent; i++)
      {
         result = result * base;
      }
      return result;
   }
```

我這邊的結果是 `OK (11 tests)`。

目前的已知缺口（下一輪要抓的）：指數是負數時迴圈一次都不會跑，所以會回傳 1。例如 `power(2, -1)` 現在得到 1，但你的規則是截斷成 0。這一點刻意留給下一輪的測試去發現。

貼上這一行（測試全過才會 commit）：

```
mvn clean test > q6-evidence/cycle4-green.txt 2>&1 && tail -8
q6-evidence/cycle4-green.txt && git add src/main && git commit -m "feat(q6):
power by repeated multiplication"
```

回覆「好了」，就進入第 5 輪：負指數。

第 47 則

學生 **prompt**：

〔附圖：第 4 輪綠燈與 feat commit 畫面〕

好了

〔Agent 動作〕確認 git log 最新為 `319bb4e feat(q6): power by repeated multiplication`；在 CalcTest.java 新增 `testPowerNegativeExponentTruncates`（未修改 Calc.java）；沙盒執行取得紅燈；更新 agent-run-log.txt。

Claude 回應：

第 4 輪完成，實作 commit 是 `319bb4e`。

Q6 第 **5** 輪：紅燈（負指數）。

我只在 `CalcTest.java` 加了一個測試：

```
   // Q6 cycle 5 (agent-written): decision (c) - negative exponent.
   // The mathematical result 1 / base^|exponent| is truncated toward zero to
an int.
   @Test public void testPowerNegativeExponentTruncates()
   {
      assertTrue ("Calc power incorrect for (2, -1)",   0 == Calc.power (2,
-1));   //  0.5   -> 0
      assertTrue ("Calc power incorrect for (-2, -1)",  0 == Calc.power (-2,
-1));  // -0.5   -> 0
      assertTrue ("Calc power incorrect for (1, -3)",   1 == Calc.power (1,
-3));   //  1     -> 1
      assertTrue ("Calc power incorrect for (-1, -3)", -1 == Calc.power (-1,
-3));  // -1     -> -1
      assertTrue ("Calc power incorrect for (-1, -2)",  1 == Calc.power (-1,
-2));  //  1     -> 1
   }
```

我跑出來的失敗輸出：

```
1) testPowerNegativeExponentTruncates(CalcTest)
java.lang.AssertionError: Calc power incorrect for (2, -1)
Tests run: 12,  Failures: 1
```

上一輪的迴圈遇到負指數一次都不跑，回傳 1，所以第一個斷言就抓到了。

⚠️ 這一輪請你特別審查，因為有一部分是我推論的：你只選了「整數截斷，例如 `power(2,-1) == 0`」。後面四個斷言是我從數學推論出來的，你沒有明確說過：

輸入	數學值	截斷後	為什麼
`(-2, -1)`	-0.5	**0**	往 0 截斷，跟 Java 整數除法一致；如果往下取整會是 -1
`(1, -3)`	1	**1**	1 的任何次方都是 1，截斷不影響
`(-1, -3)`	-1	**-1**	奇數次方保留負號

輸入	數學值	截斷後	為什麼
`(-1, -2)`	1	**1**	偶數次方變正

如果你同意，報告會寫成「agent 推論、經學生審查同意」。如果不同意，例如希望底數是 ±1 時也一律回傳 0，告訴我，我會修改測試後再請你審查一次，絕不會在你 commit 之後偷偷改。

同意的話，貼上這一行：

```
mvn clean test > q6-evidence/cycle5-red.txt 2>&1; grep -E "Tests run:|<<<|incorrect" q6-evidence/cycle5-red.txt | tail -4; git add src/test && git commit -m "test(q6): negative exponent truncates toward zero"
```

回覆「好了」。

第 48 則

學生 **prompt**：

〔附圖：第 5 輪紅燈與 test commit 畫面〕

好了

〔Agent 動作〕確認 git log 最新為 `d6bf46a test(q6): negative exponent truncates toward zero`；在 Calc.java 的 power 加入負指數處理（未修改測試檔）；沙盒執行 OK (12 tests)；更新 agent-run-log.txt。

Claude 回應：

第 5 輪的測試 commit `d6bf46a` 已確認，表示你同意了我推論的那四個斷言，報告會照實記錄。

Q6 第 **5** 輪：綠燈。只改了 `Calc.java`，在迴圈前面加上負指數的處理：

```
   // Q6 cycle 5 (agent-written): negative exponent -> 1 / base^|exponent| truncated toward zero.
   // Only base 1 and -1 give a non-zero integer; every other base truncates to 0.
   static public int power (int base, int exponent)
   {
      if (exponent < 0)
      {
         if (base == 1)  return 1;
         if (base == -1) return (exponent % 2 == 0) ? 1 : -1;
         return 0;
      }
      int result = 1;
      ...(迴圈不變)
```

我這邊的結果是 `OK (12 tests)`。

為什麼不直接算 **`1.0 / Math.pow(...)`** 再轉 **int**：分母只要絕對值大於 1，結果就落在 -1 到 1 之間（不含兩端），截斷後一定是 0。只有底數是 ±1 時結果才是整數。直接判斷比較精確，也不會有浮點數誤差。

目前的已知缺口（下一輪要抓的）：`power(0, -1)` 現在走到 `return 0`，回傳 0，但你選的規則是拋出 ArithmeticException。

貼上這一行：

```
mvn clean test > q6-evidence/cycle5-green.txt 2>&1 && tail -8
q6-evidence/cycle5-green.txt && git add src/main && git commit -m "feat(q6):
negative exponent truncates toward zero"
```

回覆「好了」，就進入第 6 輪：`power(0, -1)`。

第 49 則

學生 **prompt**：

〔附圖：第 5 輪綠燈與 feat commit 畫面〕

好了

〔Agent 動作〕確認 git log 最新為 `57c597d feat(q6): negative exponent truncates toward zero`；在 CalcTest.java 新增 `testPowerZeroToNegativeThrows`（未修改 Calc.java）；沙盒執行取得紅燈；更新 agent-run-log.txt。

Claude 回應：

第 5 輪完成，實作 commit 是 `57c597d`。

Q6 第 6 輪：紅燈（`power(0, -1)`）。

我只在 `CalcTest.java` 加了一個測試：

```
   // Q6 cycle 6 (agent-written): student decision - power(0, negative) is
1/0, so it throws
   @Test(expected = ArithmeticException.class)
   public void testPowerZeroToNegativeThrows()
   {
      Calc.power(0, -1);
   }
```

我跑出來的失敗輸出：

```
1) testPowerZeroToNegativeThrows(CalcTest)
java.lang.AssertionError: Expected exception: java.lang.ArithmeticException
Tests run: 13,  Failures: 1
```

這次紅燈的原因，正是上一輪預告的缺口：底數 0 遇到負指數時走到 `return 0`，沒有拋例外。跟 modulo 第 1 輪不同，這是行為上的紅燈，例外規格已經真正被檢查到了。這次證明「測試能抓到不拋例外的實作」不需要另外示範。

審查後貼上這一行：

```
mvn clean test > q6-evidence/cycle6-red.txt 2>&1; grep -E "Tests
run:|<<<|Expected" q6-evidence/cycle6-red.txt | tail -4; git add src/test &&
git commit -m "test(q6): power(0, negative) throws ArithmeticException"
```

回覆「好了」。

第 50 則

學生 **prompt**：

〔附圖：與上一則相同的第 5 輪畫面〕

好了

〔Agent 動作〕檢查學生 repo：最新 commit 仍是 `57c597d`，CalcTest.java 仍為 modified，`q6-evidence/` 裡沒有 cycle6-red.txt。

Claude 回應：

第 6 輪的測試還沒 **commit**。你傳的截圖跟上一次一樣，最後一行還是第 5 輪的 `57c597d`，`q6-evidence/` 裡也還沒有 `cycle6-red.txt`。看起來是上一則訊息的指令還沒貼上。

我這邊看到 `CalcTest.java` 仍是 modified 狀態，裡面就是新加的 `testPowerZeroToNegativeThrows`。請貼上這一行：

```
mvn clean test > q6-evidence/cycle6-red.txt 2>&1; grep -E "Tests
run:|<<<|Expected" q6-evidence/cycle6-red.txt | tail -4; git add src/test &&
git commit -m "test(q6): power(0, negative) throws ArithmeticException"
```

畫面上應該會看到：

- `Expected exception: java.lang.ArithmeticException`
- `Tests run: 13, Failures: 1`
- 一行 `[main xxxxxxx] test(q6): power(0, negative) throws ArithmeticException`

看到這三樣後回覆「好了」，我會先檢查 git log 再寫實作。

第 51 則

學生 **prompt**：

〔附圖：第 6 輪紅燈與 test commit 畫面〕

好了

〔Agent 動作〕確認 git log 最新為 `a18c04f test(q6): power(0, negative) throws ArithmeticException`；在 Calc.java 的負指數區塊加入 `base == 0` 時拋例外（未修改測試檔）；沙盒執行 OK (13 tests)；更新 agent-run-log.txt。

Claude 回應:

測試 commit `a18c04f` 已確認, 紅燈也正確:`Expected exception: java.lang.ArithmeticException`。

Q6 第 **6** 輪:綠燈。只在 `Calc.java` 的負指數區塊最前面加了兩行:

```
        // Q6 cycle 6 (agent-written): 0 to a negative power is 1/0 -> student chose to throw
        if (base == 0)  throw new ArithmeticException("zero to a negative power");
```

我這邊的結果是 `OK (13 tests)`。這一輪我自己決定的只有例外訊息的文字, 測試沒有檢查訊息內容。

貼上這一行:

```
mvn clean test > q6-evidence/cycle6-green.txt 2>&1 && tail -8 q6-evidence/cycle6-green.txt && git add src/main && git commit -m "feat(q6): power(0, negative) throws ArithmeticException"
```

回覆「好了」, 就進入第 7 輪, 也是最後一輪:溢位。

第 52 則

學生 **prompt**:

〔附圖:第 6 輪綠燈與 feat commit 畫面〕

好了

〔Agent 動作〕確認 git log 最新為 `4dc9d64 feat(q6): power(0, negative) throws ArithmeticException`;在 CalcTest.java 新增 `testPowerOverflowThrows`(未修改 Calc.java);沙盒執行取得紅燈;更新 agent-run-log.txt。

Claude 回應:

第 6 輪完成, 實作 commit 是 `4dc9d64`。

Q6 第 **7** 輪:紅燈(溢位, 最後一輪)。

我只在 `CalcTest.java` 加了一個測試:

```
   // Q6 cycle 7 (agent-written): student decision - overflow throws ArithmeticException.
   // 2^31 = 2147483648 is one more than Integer.MAX_VALUE (2147483647).
   @Test(expected = ArithmeticException.class)
   public void testPowerOverflowThrows()
   {
      Calc.power(2, 31);
   }
```

我跑出來的失敗輸出:

```
1) testPowerOverflowThrows(CalcTest)
java.lang.AssertionError: Expected exception: java.lang.ArithmeticException
Tests run: 14,  Failures: 1
```

目前的迴圈會默默溢位：`2^31` 會變成 `-2147483648`，不會拋例外，所以是紅燈。

請你審查時注意一個弱點，可以寫進審查意見，也是 Q7 的好素材：這個測試只檢查一個輸入。像 `if (exponent >= 31) throw ...` 這種「只對測試輸入有效」的作弊實作也能通過，但它抓不到 `power(10, 10)` 這種溢位。我打算在實作裡用 `Math.multiplyExact`，它對所有溢位都有效。真正能驗證這一點的是 Q7 的性質測試。要不要現在就多加幾個溢位案例，由你決定。如果要，告訴我，我會修改測試後在你 **commit** 之前請你再審一次。

沒問題的話，貼上這一行：

```
mvn clean test > q6-evidence/cycle7-red.txt 2>&1; grep -E "Tests
run:|<<<|Expected" q6-evidence/cycle7-red.txt | tail -4; git add src/test &&
git commit -m "test(q6): power overflow throws ArithmeticException"
```

回覆「好了」。

第 53 則

學生 **prompt**：

〔附圖：第 7 輪紅燈與 test commit 畫面；學生未附文字〕

〔Agent 動作〕確認 git log 最新為 `369b62d test(q6): power overflow throws ArithmeticException`；把 Calc.java 迴圈中的乘法改為 `Math.multiplyExact`（未修改測試檔）；沙盒執行 OK (14 tests)；更新 agent-run-log.txt。

Claude 回應：

測試 commit `369b62d` 已確認。

Q6 第 7 輪：綠燈，modulo 和 power 的 TDD 全部完成。

只改了 `Calc.java` 迴圈裡的一行：

```
        // Q6 cycle 7 (agent-written): multiplyExact throws
ArithmeticException on any int overflow
        result = Math.multiplyExact(result, base);
```

我這邊的結果是 `OK (14 tests)`。

`Math.multiplyExact` 是 Java 內建方法，每次相乘都會檢查有沒有超過 int 範圍，超過就拋 `ArithmeticException`。它對所有溢位都有效，不是只針對 `2^31`。不過目前的測試只證明了一個輸入，這一點留給 Q7 驗證。

請貼上這一行（綠燈後 commit，再印出完整 git log 供報告使用）：

```
mvn clean test > q6-evidence/cycle7-green.txt 2>&1 && tail -8
q6-evidence/cycle7-green.txt && git add src/main && git commit -m "feat(q6):
```

```
power throws on int overflow (Math.multiplyExact)" && git log --oneline > q6-evidence/git-log.txt && cat q6-evidence/git-log.txt
```

接著請你截兩張圖，這是必交項目：

1. 跑 `mvn clean test`，截到 `Tests run: 14, Failures: 0, Errors: 0` 和 `BUILD SUCCESS`。這是「全部測試通過」的截圖。
2. 剛才那行指令最後印出的完整 `git log`。

回覆「好了」後，我會開始：

- 整理 Q6 的違規紀錄和「agent 未經詢問的決定」清單
- Q7：開一個乾淨的分支，只用你的測試重新生成 Calc，再用性質測試檢查它是不是只對測試輸入有效
- Q2、Q3：GitHub CI 步驟
- 最後把全部整理成一份報告

第 54 則（Q6 收尾、Q7、報告）

學生 **prompt**：

〔附圖：第 7 輪綠燈（Tests run: 14, Failures: 0）與完整 git log 畫面；學生未附文字〕

〔Agent 動作〕 1. 檢查 Q6 期間每個 commit 修改的檔案：每個 test(q6) commit 只改 CalcTest.java，每個 feat(q6) commit 只改 Calc.java；測試檔被刪除的行數為 0。 2. Q7：把 CalcTest.java 的 `//` 註解全部拿掉，啟動一個全新、沒有對話脈絡的 sub-agent，只給它測試並要求寫出 Calc.java（逐字 prompt 與回應見附錄）。 3. Q7：撰寫刻意「只對測試輸入有效」的作弊對照組 Calc，以及 9 個性質測試／蛻變測試 `CalcPropertyTest.java`；對 Q6 最終版、sub-agent 版、作弊版三種 Calc 各跑 14 個範例測試與 9 個性質測試；量測 `power(1, Integer.MAX_VALUE)` 的執行時間（約 1.4 秒）。 4. 撰寫 `REPORT.md`，並把報告、Q6 截圖、Q7 檔案、Q1 截圖寫入學生的 calc-tdd 資料夾（未 commit）。

Claude 回應：

Q6 和 Q7 做完了，報告也寫好了。所有檔案都已放進你的 `calc-tdd` 資料夾：`REPORT.md`、`q6-evidence/`（每輪紅燈和綠燈的輸出紀錄、我已命名好的截圖）、`q7/`（性質測試和 agent 的逐字紀錄）、`q1-screenshots/`。

Q6 結果：共 7 輪，14 個測試全部通過。我用 git 檢查過，每個測試 commit 只改測試檔，每個實作 commit 只改 `Calc.java`，順序完全交替，已經 commit 的測試從來沒有被改過或刪過。

我（agent）有一個違規，已經照實寫在報告 §2.4。規則 (c) 要求四個決定都要在「任何程式碼」寫出來之前，先用測試定下來。但我選了一輪處理一個決定，結果「modulo 負數運算元」和「負指數」這兩個測試，是在 modulo 和 power 已經有程式碼之後才寫的。另外還有兩個較輕的問題，以及 10 項我沒先問你就做的決定，例如負指數時 `(-2,-1)` 回傳多少，是我推論後交給你審查的。

Q7 結果：

Calc 版本	原本 14 個範例測試	9 個性質測試
全新 agent 只看測試寫出來的版本	全過	全過，沒有作弊
我故意寫的「只對測試輸入有效」版本	也全過	3 個失敗
我們 Q6 的最終版	全過	1 個失敗：`power(1, 最大整數)` 要跑約 1.4 秒，結果正確但很慢

結論是：只靠範例測試，分不出通用的實作和只對測試輸入有效的實作；性質測試分得出來。

還要你自己完成的部分：

1. 把新檔案 **commit** 進 **repo**：

   ```
   git add REPORT.md q6-evidence q7 q1-screenshots && git commit -m "docs: report, Q6 evidence, Q7 property tests"
   ```

2. 寫 **Q1** 的敘述：填在報告 §1.3。git 紀錄的事實表我已經整理在 §1.1。§1.4 照實寫了我在 Q1 幫過哪些忙（解釋流程、審查你寫的測試、教 JUnit 語法），請確認內容沒錯。

3. 整理 **Q1** 截圖：`q1-screenshots/unsorted-01` 到 `unsorted-10` 我沒辦法確定每張對應哪一輪，請你自己對照命名。

4. 匯出這段對話作為 AI 使用紀錄。作業要求逐字的 prompt 和完整回應，我沒辦法替你匯出。

5. **Q2**、**Q3**：

   - 在 GitHub 建一個空的 repo，然後執行 `git remote add origin <URL>` 和 `git push -u origin main`。
   - 到 Actions 頁面截綠燈的圖，再下載 `jacoco-report` 截覆蓋率報告的圖。
   - 故意弄壞一個地方，例如把 `subtract` 改成 `a + b`，push 後截紅燈的圖，自己修好再 push 一次。
   - 用同樣的方式再弄壞一次，只告訴我「CI 失敗了，幫我修」。看我是改程式還是改測試，用來跟你自己的修法比較。

另外，Q7 找到的 `power` 效能問題可以再做一輪 TDD 來修（第 8 輪），流程跟前面一樣：先寫紅燈測試，你 commit 後我再實作。要做的話告訴我。

第 55 則

學生 **prompt**：

1-3內容已完成 接下來還有哪些地方需要修改q6的意見審查 我在之前對話中已經審核過 為什麼只有第一輪有內容 並且我應該在做完q2 q3後才匯出對話紀錄嗎

〔Agent 動作〕檢查學生 repo 狀態與 REPORT.md 中的審查意見段落；備份 REPORT.md 後，把 §2.3「第 2 到 7 輪：_____」一行改成依對話紀錄整理的表格（其他內容未動）。

Claude 回應：

為什麼審查意見只有第 **1** 輪有內容

我寫報告時是照這段對話的紀錄寫的。第 2 到 7 輪你的回覆都是「好了」加上截圖，只有第 1 輪你提出了具體意見（紅燈來自編譯錯誤，還沒驗證到例外型別）。所以我只把第 1 輪寫成「審查意見」，其他輪沒有替你編寫。

這不是只為了誠實，也很實際：你要附上逐字的對話紀錄。如果報告寫了一段審查意見，但對話裡找不到，用 AI 評分時很容易被比對出來。「好了」加上你親自 commit，本身就是有效的審查：你看過測試、同意它，然後才 commit。這樣照實寫就好。

我剛改了什麼

我已直接修改你資料夾裡的 `REPORT.md`，只動 §2.3 原本空著的「第 2 到 7 輪」那一行，其他內容都沒碰。修改前的備份我另外存了一份，不在你的資料夾裡。

現在那裡是一張表，每一輪列出：

- 我當時請你特別注意的地方
- 你的處理方式（都是同意、沒有修改）
- 你的 test commit hash

其中有兩輪值得你自己補幾句：

- 第 **5** 輪：4 個斷言是我推論的，你同意了。可以寫你當時為什麼覺得合理。
- 第 **7** 輪：我提醒過只測一個輸入是弱點，你沒有要求加案例。後來 Q7 證實了這個弱點，可以寫你現在的看法。

表的最後一欄是「事後補充」，明確標示是寫報告時才加的想法。你可以用中文寫，需要的話我再幫你翻成英文。

對話紀錄什麼時候匯出

等 Q2、Q3 做完再匯出，只匯出一次。原因：

- Q2 後半要我當 agent 修壞掉的 build，那段 prompt 和回應也必須放進 AI 使用紀錄。
- 這次的提問也算 AI 使用，最後一次匯出就會全部包含。

如果怕對話不見，現在可以先匯出一份備份，但最後交的一定要是 Q2、Q3 做完後的完整版本。

另外提醒一件事

`REPORT.md`、`q6-evidence/`、`q7/` 目前還沒 commit。你填完 Q1 敘述後執行：

```
git add REPORT.md q6-evidence q7 q1-screenshots && git commit -m "docs:
report, Q6 evidence, Q7 property tests"
```

做完再 push 到 GitHub 開始 Q2。Push 之後把 Actions 頁面的結果貼給我，我們接著做「弄壞 build、交給 agent 修」那一段。

第 56 則

學生 **prompt**：

如果我把q1的截圖都放在report中 並把原本的q1-screenshots刪除了 可以嗎 我應該如何寫git

〔Agent 動作〕檢查學生 repo：q1-screenshots 已不存在且從未被 commit；REPORT.md 中沒有內嵌圖片、§1.3 仍為空白、仍有三處引用 q1-screenshots。

Claude 回應：

可以刪，git 也不用特別處理，但我在你資料夾裡看到一個需要先確認的問題。

git 方面不用做任何事

`q1-screenshots/` 從來沒有被 commit 過。之前的狀態是 untracked，現在你的資料夾裡也已經沒有它了。git 從頭到尾都不知道這個資料夾存在，所以刪掉不會留下任何紀錄，也不需要 `git rm`。

先確認你改的是哪一份報告

我查看了 `calc-tdd/REPORT.md`，裡面：

- 沒有任何截圖。Markdown 本身不能把圖片「放進」檔案，只能寫 `![說明](路徑)` 連到圖檔。刪掉圖檔後連結就會失效。
- **§1.3** 的 **Q1** 敘述還是空白的，最後修改時間是 10/23。

所以你說「已完成、截圖放進報告」的應該是另一份檔案，例如 Word 或 Google Docs。如果是這樣：

- 截圖已經貼進那份 Word/PDF 的話，刪掉 `q1-screenshots/` 沒問題。
- 但 repo 裡的 `REPORT.md` 有三個地方還指向 `q1-screenshots/`（第 40、41、329 行），而且 Q1 敘述是空的。

你要二選一：

1. 以 **Word/PDF** 為正式報告：在 `REPORT.md` 開頭註明「正式報告見 XXX.pdf」，或乾脆不 commit `REPORT.md`，改 commit 那份 PDF。
2. 以 **`REPORT.md`** 為正式報告：把你寫好的 Q1 敘述也貼進 `REPORT.md` 的 §1.3，再把那三行的 `q1-screenshots/...` 改成「見 PDF 報告」或刪掉。

commit 指令

報告放在 repo 外面（例如交到 E3 的 Word/PDF）時，執行：

```
git add q6-evidence q7 REPORT.md
git commit -m "docs: Q6 evidence and Q7 property tests"
```

報告的 PDF 也想放進 repo 的話，把它複製到 `calc-tdd` 資料夾，再執行：

```
git add q6-evidence q7 REPORT.md 報告檔名.pdf
git commit -m "docs: report, Q6 evidence, Q7 property tests"
```

commit 之前可以用 `git status` 確認，清單裡不該出現 `target/`，因為已經被 `.gitignore` 排除了。

你的正式報告是哪一種檔案、放在哪裡？告訴我之後，我可以幫你把 `REPORT.md` 那三行引用改好，或把它跟正式報告同步。

第 57 則

學生 **prompt**：

以 Word/PDF 為正式報告，改 commit 那份 PDF。 但這樣的格式有符合作業要求嗎

Claude 回應：

可以，Word/PDF 報告加上 GitHub repo 是這類作業最常見的繳交方式。但要先說明一點：作業 PDF 我一開始就沒能完整讀到內容，所以不確定老師有沒有規定特定格式，例如只能交一個 PDF、要交 zip，或要附 repo 連結。請自己翻一下作業 PDF 最後的 Submit 段落確認。

依你貼給我的繳交清單，每一項的安排如下：

要求	放在哪	注意
你的測試	PDF 附錄（文字）＋ repo	程式碼要用文字貼，不要用截圖
最終版 Calc	PDF 附錄（文字）＋ repo	同上
git log	PDF 附錄（文字）	貼 `git log --oneline` 的輸出
全部測試通過的截圖	PDF 內嵌圖片	用 `Tests run: 14, Failures: 0` 那張
敘述（Q1、Q6）	PDF 正文	
AI 使用紀錄（逐字 prompt＋完整回應）	另附檔案，PDF 裡註明檔名	對話很長，放進 PDF 正文會很亂

老師用 **AI** 評分時特別要注意的一點

AI 讀 PDF 時，文字讀得最準，截圖裡的字容易讀錯或漏讀。所以程式碼、git log、紅燈訊息（例如 `Expected exception: java.lang.ArithmeticException`）都要有文字版，截圖只當佐證。我之前整理的 `REPORT.md` 裡已經有這些文字，可以直接複製到 Word。

git 怎麼寫

把 PDF 複製到 `calc-tdd` 資料夾後執行：

```
git add q6-evidence q7 你的報告.pdf
git commit -m "docs: final report (PDF), Q6 evidence, Q7 property tests"
git status
```

`REPORT.md` 沒有 add 進去，所以不會進入 repo。它留在資料夾裡當參考也沒關係，不想要的話自己刪掉就好。

建議的時間順序

Q2 和 Q3 還沒做，報告之後還要再補內容。建議這樣排：

1. 現在先 commit `q6-evidence` 和 `q7`，PDF 等最後再放。
2. push 到 GitHub，做 Q2 和 Q3。
3. 把 Q2、Q3 的結果補進 Word。
4. 匯出完整的對話紀錄。
5. 最後輸出 PDF，commit 並 push。

PDF 做好後可以傳給我。我會照上面的表和報告的繳交清單逐項檢查有沒有缺漏，不會改你寫的內容。

第 58 則

學生 **prompt**：

```
You will use test-driven development (TDD) to extend a simple calculator,
Calc. Starter versions in Java, JavaScript, and Python are attached. Full
instructions are in the PDF.
Required:

* Question 1: add subtract, multiply, and divide one failing test at a time,
and write a narrative of each red-green-refactor cycle. Do this question
yourself, without an AI coding agent.
* Question 6: use an AI coding agent to add modulo and power, again with TDD.
The rules:
   * The agent writes a failing test first.
   * You review and commit each test before the agent implements it.
   * Your tests must define the results for negative operands, modulo by
zero, power(0, 0), and negative exponents before any code is written.
Report every time the agent broke a TDD rule, and every decision it made
without asking you.

Optional:

* Questions 2–5 cover CI, verification tools, refactoring, and fault repair.
Questions 2 and 5 now also ask you to compare your work with an agent's.
* Question 7 checks whether code an agent writes from your tests only works
for the test inputs.

Submit:
```

```
* your tests;
* your final Calc;
* your git log;
* a screenshot showing that all tests pass;
* your narratives;
* an AI usage log for every question where you used AI, with every prompt
verbatim and the agent's full responses.
```

Claude 回應:

這段要求裡沒有規定檔案格式, 只列出要交的六項內容。所以「Word/PDF 報告＋GitHub repo」符合要求, 只要六項都有就好。逐項對照:

要求	你的位置	狀態
your tests	`CalcTest.java`:Q1 的 7 個＋Q6 的 7 個; `q7/CalcPropertyTest.java`。PDF 附錄貼文字	✅ 已有
your final Calc	`Calc.java`, PDF 附錄貼文字	✅ 已有
your git log	`git log --oneline` 的文字, 最後交件前要重新輸出一次, 才會包含 Q2 的 commit	⚠️ 最後再更新
a screenshot showing that all tests pass	`c7-green-all-14-pass.png`	✅ 已有
your narratives	Q1:你寫的版本;Q6:每輪表格加審查意見	✅ 依你所說 Q1 已完成
AI usage log(每個用過 AI 的題目, 逐字 prompt＋完整回應)	對話匯出檔＋ `q7/agent-transcript.md`	⬜ 等 Q2、Q3 做完再匯出

對照原文後要特別確認的三點:

1. **Q6** 的違規 **V1** 判斷是對的。原文寫明 “define the results for negative operands, modulo by zero, power(0, 0), and negative exponents **before any code is written**”。我是一個決定做一輪, 所以「modulo 負數運算元」和「負指數」的測試, 都是在 modulo 和 power 已有程式碼之後才寫的, 報告裡如實記為違規是正確的。要求寫的是 “Report every time the agent broke a TDD rule”, 所以這條一定要保留, 不要刪掉。
2. **“negative operands”** 可能也包括 **power** 的負底數。目前 `power(-2, 3)` 這種負底數、正指數的情況沒有任何範例測試, 只有 Q7 的性質測試涵蓋到。建議在報告的違規或決定清單裡補一句:「負底數的 power 結果沒有在 Q6 的範例測試中定義」。這屬於 agent 沒有問你、也沒有寫測試的地方, 報告裡已有的 D4 可以順便擴充這一點。

3. **Q2** 要和 **agent** 比較。原文寫 "Questions 2 and 5 now also ask you to compare your work with an agent's"。所以 Q2 不能只交 CI 截圖，必須有「你修 build」和「agent 修 build」兩份結果，再加上比較。這一段的 AI prompt 和回應也要放進 AI 使用紀錄。

AI 使用紀錄要涵蓋的題目：Q1（只有說明和指導，報告 §1.4 已聲明）、Q6、Q7，以及待做的 Q2。四題都在這段對話裡，Q7 另有 `agent-transcript.md`，所以最後匯出一次完整對話即可。

下一步是 push 到 GitHub 開始做 Q2。push 完把 Actions 頁面的結果傳給我。

第 59 則

學生 **prompt**：

好的請開始告訴我如何做

〔Agent 動作〕檢查學生 repo 狀態（REPORT.md、q6-evidence、q7 未 commit；尚無 remote）。

Claude 回應：

你的 repo 目前還沒 commit 新檔案，也還沒連到 GitHub，所以從頭開始。全部都在 `calc-tdd` 資料夾的終端機裡執行。

步驟 *0*：先 *commit Q6* 和 *Q7* 的證據

```
git add q6-evidence q7
git commit -m "docs: Q6 evidence and Q7 property tests"
```

步驟 *1*：建立 *GitHub repo* 並 *push*

用 GitHub 官方的 CLI 工具 `gh` 最簡單，它會一起處理登入：

```
brew install gh
gh auth login
```

登入時依序選擇 **GitHub.com → HTTPS → Login with a web browser**，然後照畫面在瀏覽器裡按確認。

登入完成後執行：

```
gh repo create calc-tdd --private --source=. --push
```

要不要設成 **public**：老師或助教要能看到 repo，評分才看得到 CI 結果。可以用 public（把指令裡的 `--private` 改成 `--public`），或是維持 private，之後到 repo 的 Settings → Collaborators 邀請助教。依課程規定決定。

步驟 *2*：確認 *CI*（*Q2* 前半，以及 *Q3*）

1. 到 GitHub 的 repo 頁面，點上方的 **Actions**，等那次執行跑完，大約 1 到 2 分鐘。
2. 綠色 ✓：截圖，這是 Q2 的「CI 正常」證據。
3. 紅色 ✗：點進去複製錯誤訊息貼給我。CI 會跑 SpotBugs 和 100% 分支覆蓋率檢查，這兩項你的電腦都沒實際跑過，所以有可能失敗。

Q3 的截圖：

- 在那次執行的頁面最下方 **Artifacts** 下載 `jacoco-report`，解壓後打開 `index.html`，截覆蓋率表格。
- 點進 `build` 這個 job，展開「Build, test, coverage, static analysis」，找到 SpotBugs 那段輸出（例如 `BugInstance size is 0`），截圖或複製文字。

步驟 3：自己弄壞再修好 *build*（*Q2* 學生版）

```
# 1. 打開 src/main/java/Calc.java，把 subtract 的 return a - b; 改成 return a + b;
git add src/main
git commit -m "chore(q2): break the build on purpose (subtract)"
git push
```

到 Actions 頁面截紅色 ✗ 的圖，並點進去截到哪個測試失敗。

接著自己修好：

```
# 2. 把 subtract 改回 return a - b;
git add src/main
git commit -m "fix(q2): restore subtract"
git push
```

再截一次綠色 ✓。記下你怎麼判斷要改哪裡、花了多久，比較時會用到。

步驟 4：再弄壞一次，交給 *agent* 修（*Q2 agent* 版）

這一步不要告訴我你改了什麼，這樣比較才公平。

- 自己選一個弄壞的方式，commit 並 push，讓 CI 變紅。
- 最能觀察 agent 行為的弄壞方式，是改 **Calc** 的某個行為，讓程式和測試看起來都可能是「錯的那一方」。這樣才看得出 agent 會修程式，還是去改測試來配合。

CI 變紅後，傳給我一則 prompt，例如：

> CI 失敗了，請幫我修好讓 build 通過。（貼上 Actions 頁面的錯誤訊息）

我會當 agent 處理，你觀察三件事：

1. 我改的是程式還是測試？
2. 我有沒有先問你？
3. 我的修法和你在步驟 3 的修法有什麼不同？

這段的 prompt 和回應都會留在對話紀錄裡，就是 Q2 的 AI 使用紀錄。

先完成步驟 0 到 2，把 Actions 的結果傳給我。

第 60 則

學生 **prompt**：

```
Enter the code displayed in the app or on the device you're signing in to.
Never use a code sent by someone else. 如果沒有收到 該怎麼收
```

Claude 回應:

這個 code 不會寄到你的手機或 email, 它就顯示在你的終端機裡。

執行 `gh auth login` 並選好選項之後, 終端機會出現類似這兩行:

```
! First copy your one-time code: ABCD-1234
Press Enter to open github.com in your browser...
```

照這樣做:

1. 在終端機找到 `one-time code` 後面那組 8 碼, 例如 `ABCD-1234`。
2. 在終端機按 **Enter**, 瀏覽器會自動開到 GitHub 的頁面。
3. 把那組 code 輸入瀏覽器頁面的格子, 按 **Continue**, 再按 **Authorize GitHub CLI**。
4. 回到終端機, 看到 `✓ Logged in as 你的帳號` 就成功了。

如果終端機的 **code** 被洗掉、找不到了:在終端機按 `Ctrl + C` 取消, 重新執行 `gh auth login`, 會產生一組新的 code。

注意:瀏覽器頁面上那句「Never use a code sent by someone else」是安全提醒。只輸入你自己終端機顯示的 code, 任何人傳給你的 code 都不要輸入。

登入成功後, 執行:

```
gh repo create calc-tdd --private --source=. --push
```

第 61 則

學生 **prompt**:

```
Press Enter to open https://github.com/login/device in your browser...


gh repo create calc-tdd --public --source=. --push 好像沒看到
```

Claude 回應:

你還停在登入步驟, 所以 `gh repo create` 那行沒有作用。登入要先完成, 才能建立 repo。

code 在「**Press Enter...**」那一行的上一行, 長這樣:

```
! First copy your one-time code: ABCD-1234
Press Enter to open https://github.com/login/device in your browser...
```

照這個順序做:

1. 先按 **`Ctrl + C`**, 把目前卡住的登入取消。你剛才打的 `gh repo create` 已經混進等待中的輸入, 乾脆重來比較乾淨。

2. 重新執行：

   ```
   gh auth login --web
   ```

3. 看終端機上的 `First copy your one-time code: XXXX-XXXX`，把這 8 碼記下來或複製起來。

4. 按 **Enter**，瀏覽器會開到 `github.com/login/device`。

5. 在瀏覽器輸入那 8 碼，按 **Continue**，再按 **Authorize GitHub CLI**。如果 GitHub 要你輸入密碼或做驗證，照做即可。

6. 回到終端機，等它顯示 `✓ Logged in as 你的帳號`。

7. 看到這一行之後，再執行：

   ```
   gh repo create calc-tdd --public --source=. --push
   ```

完成後終端機會印出 repo 網址，例如 `https://github.com/你的帳號/calc-tdd`。如果中間哪一步的畫面跟上面描述的不一樣，把終端機的文字貼給我。

第 62 則

學生 **prompt**：

```
ning@Mac calc-tdd %    gh repo create calc-tdd --public --source=. --push
To get started with GitHub CLI, please run:  gh auth login
Alternatively, populate the GH_TOKEN environment variable with a GitHub API
authentication token.
```

Claude 回應：

你還沒登入成功，所以 `gh repo create` 被拒絕了。登入一定要走完，終端機出現 `Logged in` 才算數。

請照下面一步一步做，每一步看到預期的畫面才做下一步：

1. 執行登入：

```
gh auth login --web --git-protocol https
```

2. 終端機會出現：

```
! First copy your one-time code: XXXX-XXXX
Press Enter to open https://github.com/login/device in your browser...
```

把 `XXXX-XXXX` 那 8 碼記下來，然後按 **Enter**。

3. 在瀏覽器裡：

- 頁面會開到 `github.com/login/device`。如果沒有自動打開，就自己在瀏覽器輸入這個網址。

- 沒登入 GitHub 的話先登入。
- 輸入剛才的 8 碼，按 **Continue**。
- 按綠色的 **Authorize GitHub CLI**。
- 看到「Congratulations, you're all set!」就完成了。

4. 回到終端機，等它自己跑完，最後會出現：

```
✓ Authentication complete.
✓ Logged in as 你的帳號
```

5. 確認登入狀態：

```
gh auth status
```

顯示 `Logged in to github.com account 你的帳號` 才代表成功。

6. 最後才執行：

```
gh repo create calc-tdd --public --source=. --push
```

如果在第 2 步按了 Enter 之後終端機就結束，或在第 4 步出現錯誤，把終端機的完整文字貼給我。

第 63 則

學生 **prompt**：

```
ning@Mac calc-tdd % gh repo create calc-tdd --public --source=. --push
✓ Created repository ning415415/calc-tdd on github.com
  https://github.com/ning415415/calc-tdd
✓ Added remote https://github.com/ning415415/calc-tdd.git
Enumerating objects: 200, done.
Counting objects: 100% (200/200), done.
Delta compression using up to 8 threads
Compressing objects: 100% (145/145), done.
Writing objects: 100% (200/200), 5.29 MiB | 4.38 MiB/s, done.
Total 200 (delta 56), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (56/56), done.
To https://github.com/ning415415/calc-tdd.git
 * [new branch]      HEAD -> main
branch 'main' set up to track 'origin/main'.
✓ Pushed commits to https://github.com/ning415415/calc-tdd.git
已經成功登入
```

〔Agent 動作〕透過 GitHub 公開 API 查詢 Actions：CI #1 completed / success；各步驟皆 success；artifact jacoco-report 已產生。

Claude 回應：

Repo 建好了，第一次 CI 也已經跑完，結果是綠燈 ✓。我透過 GitHub 的公開 API 確認了這次執行：

步驟	結果
Build, test, coverage, static analysis（`mvn verify`）	✅ success
Upload coverage report	✅ success，產生 `jacoco-report`

這代表在 CI 的 JDK 17 上，14 個測試全部通過，JaCoCo 的 100% 分支覆蓋率和 SpotBugs 靜態分析也都通過了。你的 Mac 跑不了 SpotBugs，這是第一次真正驗證它。

接著請截 **Q2** 和 **Q3** 的圖：

1. **CI** 綠燈（**Q2**）：打開 https://github.com/ning415415/calc-tdd/actions/runs/40151608733 ，截有綠色 ✓ 的頁面。
2. **SpotBugs** 結果（**Q3**）：同一頁點左邊的 **build**，展開 **Build, test, coverage, static analysis**。往下找 `spotbugs` 開頭的那幾行，例如 `BugInstance size is 0` 或 `Total bugs: 0`，截圖。
3. 覆蓋率報告（**Q3**）：同一頁最下面 **Artifacts** 點 `jacoco-report` 下載，解壓後用瀏覽器打開 `index.html`，截表格。點進 `Calc` 可以看到每一行被測試覆蓋的顏色，也一起截。

截完就做步驟 **3**：自己弄壞再修好 **build**：

```
# 把 src/main/java/Calc.java 的 subtract 改成 return a + b;  然後：
git add src/main
git commit -m "chore(q2): break the build on purpose (subtract)"
git push
```

到 Actions 頁面截紅色 ✗ 和失敗的測試名稱，再改回 `return a - b;`：

```
git add src/main
git commit -m "fix(q2): restore subtract"
git push
```

確認又變回綠燈並截圖。

做完後進行步驟 **4**：自己挑一種弄壞的方式，不要告訴我改了什麼。push 讓 CI 變紅後，傳給我一句像「CI 失敗了，請幫我修好讓 build 通過」的話，附上錯誤訊息。

第 64 則

學生 **prompt**：

〔附圖：CI #1 Summary 頁〕

沒有找到這段
`SpotBugs 結果（Q3）：同一頁點左邊的 build，展開 Build, test, coverage, static analysis。往下找 `spotbugs` 開頭的那幾行，例如 `BugInstance size is 0` 或 `Total bugs: 0`，截圖。`

〔Agent 動作〕嘗試以 GitHub API 下載 CI 記錄（需要 repo 管理員權限，失敗）。

Claude 回應：

你截的是 **Summary** 頁，SpotBugs 的輸出在 job 的詳細記錄裡，要再點進去一層。有兩種找法：

方法 **A**：在網頁上找

1. 點左邊或中間的 **build**（綠色 ✓ 那一格）。
2. 會出現步驟清單，點 **Build, test, coverage, static analysis** 展開，裡面是幾百行 Maven 輸出。
3. 右上角有 **Search logs** 搜尋框，輸入 `spotbugs`。
4. 應該會找到類似這幾行，截這一段：

   ```
   [INFO] --- spotbugs:4.8.6.6:spotbugs (spotbugs) @ calc-tdd ---
   [INFO] --- spotbugs:4.8.6.6:check (default) @ calc-tdd ---
   [INFO] BugInstance size is 0
   [INFO] Error size is 0
   ```

   再搜尋 `coverage`，會找到 `All coverage checks have been met.`，這行也截下來，它是 JaCoCo 的證據。

方法 **B**：用終端機（比較快，還能存成文字檔當證據）

你剛登入了 `gh`，所以可以直接在 `calc-tdd` 資料夾執行：

```
gh run view 40151608733 --log > q2q3-evidence-ci-run1.txt
grep -i -E "spotbugs|BugInstance|Error size|coverage checks|Tests run:|BUILD"
q2q3-evidence-ci-run1.txt
```

第一行把整份 CI 記錄存成檔案，第二行只把 SpotBugs、JaCoCo、測試結果的關鍵行挑出來顯示。把第二行的輸出貼給我，我幫你確認內容對不對。

GitHub 的記錄需要 repo 管理員權限才能下載，我這邊看不到完整記錄，只確認得到這個步驟的狀態是 success。所以實際文字要靠你截圖或用方法 B 取得。

如果搜尋 `spotbugs` 找不到任何一行，代表它根本沒執行，那也要告訴我，表示 `pom.xml` 的設定有問題。

第 65 則

學生 **prompt**：

〔附圖：CI 記錄中 SpotBugs 與 JaCoCo 段落〕

如上

Claude 回應：

這兩張截圖可以直接當 Q3 的證據：

工具	記錄中的證據	意思
JaCoCo（第 574–581 行）	`All coverage checks have been met.`	分支覆蓋率達到 100%
SpotBugs（第 1263–1271 行）	`BugInstance size is 0`、`Error size is 0`、`No errors/warnings found`	用最嚴格的設定（effort=Max、threshold=Low）分析，找不到任何問題
整體（第 1273 行）	`BUILD SUCCESS`	測試、覆蓋率、靜態分析都通過

寫報告時，第 1269–1271 行最好也貼成文字，方便評分時讀取。

Q3 只差 **JaCoCo** 報告頁的截圖：

1. 回到 Summary 頁，往下捲到 **Artifacts**，點 `jacoco-report` 下載 zip。
2. 解壓後用瀏覽器打開 `index.html`，截表格，Branches 那欄應該是 100%。
3. 點進 `Calc`，再點 `Calc.java`，會看到原始碼每一行的顏色：綠色是測到的行，黃色是分支只測到一部分，紅色是沒測到。截一張。

接著做 **Q2** 步驟 **3**：自己弄壞再修好

```
# 1. 把 src/main/java/Calc.java 的 subtract 改成 return a + b;
git add src/main
git commit -m "chore(q2): break the build on purpose (subtract)"
git push
```

到 Actions 頁面，等這次執行變成紅色 ✗。截兩張圖：Summary 頁，以及展開步驟後失敗的測試名稱和訊息（可以搜尋 `FAIL`）。然後改回來：

```
# 2. 改回 return a - b;
git add src/main
git commit -m "fix(q2): restore subtract"
git push
```

確認變回綠色 ✓ 並截圖。記下你花了多久、怎麼判斷該改哪裡。

之後是步驟 **4**：自己選一種弄壞的方式，不要告訴我改了什麼。push 讓 CI 變紅後，傳一句請我修好的話給我，並附上錯誤訊息。

第 66 則

學生 **prompt**：

〔附圖：JaCoCo 報告總覽、Calc 各方法表格〕

```
[INFO] Done SpotBugs Analysis....
[INFO]
[INFO] <<< spotbugs:4.8.6.6:check (default) < :spotbugs @ calc-tdd <<<
[INFO]
[INFO]
```

```
[INFO] --- spotbugs:4.8.6.6:check (default) @ calc-tdd ---
[INFO] BugInstance size is 0
[INFO] Error size is 0
[INFO] No errors/warnings found
```

點進 `Calc`，再點 `Calc.java`，會看到原始碼每一行的顏色：綠色是測到的行，黃色是分支只測到一部分，紅色是沒測到。截一張。這部分不確定是否是上面截圖那樣

Claude 回應：

你截的第二張是 **Calc** 類別的方法列表，還不是原始碼頁，但已經很有用。原始碼頁要再點一下：

怎麼打開原始碼頁：在這張表裡點 **`power(int, int)`** 這個藍色連結，就會跳到 `Calc.java` 的原始碼，每一行都有底色：

- 綠色：測試有執行到
- 黃色：分支只測到一部分
- 紅色：沒測到

`if` 那幾行的左邊還會有小菱形，滑鼠移上去會顯示「All 2 branches covered」。把 `power` 方法那一段截下來就好，它是 Q6 新增、分支最多的方法。

這兩張報告怎麼解讀（可以寫進 **Q3**）：

指標	結果	解讀
分支覆蓋率	**100%（0 of 12 missed）**	`power` 的 12 個分支全部被測到。這是 CI 強制的門檻。
指令覆蓋率	95%（missed 3 of 65）	唯一沒測到的是 **`Calc()`**，也就是 Java 自動產生的隱含建構子。所有方法都是 static，測試從來沒有 `new Calc()`。
行覆蓋率	14/15 行	同上，那 1 行就是建構子。這也是 `047a85a` 拿掉行覆蓋率規則的原因。
`power` 的循環複雜度（Cxty）	7	方法裡分支最多、最需要測試的地方

這可以當 Q3 的重點：覆蓋率工具把一行「看不見的程式碼」標成沒測到，你判斷它不影響行為，所以調整了規則，沒有為了衝數字而去寫無意義的測試。

SpotBugs 的文字你已經複製好了，直接貼進報告即可。

Q3 的證據到這裡就齊了，下一步是 **Q2** 步驟 **3**：把 `subtract` 改成 `return a + b;`，commit、push 看 CI 變紅，再修回來。

第 67 則

學生 **prompt**：

〔附圖：power 原始碼覆蓋率頁、終端機顯示 commit 失敗（nothing added to commit）與 push「Everything up-to-date」、CI #1 Summary 頁〕

q2的步驟三好像沒有正確執行到 沒有看到紅色

〔Agent 動作〕檢查學生 repo：Calc.java 第 15 行仍為 `return a - b;`，最新 commit 仍是 f5121f0。

Claude 回應：

你說得對，步驟三沒有執行到。原因在終端機那段訊息：

```
nothing added to commit but untracked files present
```

意思是 **`Calc.java`** 根本沒被修改，所以沒有東西可以 commit，`git push` 也只回了 `Everything up-to-date`，CI 沒有重新執行。你看到的綠燈是 12 小時前那一次（#1）。

我剛檢查了你電腦上的 `Calc.java`，第 15 行仍然是 `return a - b;`。最可能的原因是改了但沒存檔，或改到另一個檔案。

這次照這樣做：

1. 在 VS Code 打開 `src/main/java/Calc.java`，把第 15 行 `return a - b;` 改成 `return a + b;`。
2. 按 **`Cmd + S`** 存檔。存檔前分頁標籤上會有一個白色圓點，存完圓點就會消失。
3. 先在終端機確認真的改到了：

   ```
   git diff
   ```

   應該會看到：

   ```
   -       return a - b;
   +       return a + b;
   ```

   沒看到這兩行就不要往下做，表示還沒存檔。
4. commit 並 push：

   ```
   git add src/main
   git commit -m "chore(q2): break the build on purpose (subtract)"
   git push
   ```

   這次 commit 應該會出現 `1 file changed, 1 insertion(+), 1 deletion(-)`。

5. 到 GitHub 的 **Actions** 頁面，會看到一個新的執行 **#2**，標題是 `chore(q2): break the build...`。等它跑完變成紅色 ✗。

紅燈要截的圖：

- Actions 列表或 Summary 頁上的紅色 ✗。
- 點進 **build**，展開 **Build, test, coverage, static analysis**，搜尋 `FAIL` 或 `Tests run`，截到 `testSubtract` 失敗的訊息。
- 另外記下從開始修到變回綠燈花了多久。

另外，你截的 JaCoCo 原始碼頁正確，就是要那張。`power` 每個 `if` 旁邊都是綠色菱形，滑鼠移上去顯示 `All 2 branches covered`，Q3 的證據齊了。

Summary 頁下方的「Node.js 20 is deprecated」是 GitHub 對 `actions/checkout@v4` 這類官方元件的提醒，跟你的程式無關，不影響結果。

第 68 則

學生 **prompt**：

〔附圖：Actions 列表顯示 CI #2 紅燈、CI #2 testSubtract 失敗記錄〕

接下來開始修嗎

Claude 回應：

對，現在由你自己修。這是 Q2 裡「你的修法」那一半，所以我不動你的檔案，只說明步驟。

你截到的紅燈：CI #2（commit `7c999df`）是 `Tests run: 14, Failures: 2`。截圖裡看得到 `testSubtract`，往下捲應該還有第二個：**`testSubtractNegative`**。兩個 subtract 測試都抓到了這個錯，`2+3=5` 不等於 `-1`。把第二個也截下來，報告可以寫「一個錯誤被兩個測試抓到」。

修的步驟：

1. 先記下開始時間。
2. 照你自己的判斷找到錯的地方並改回來。寫報告時要描述你的判斷過程，例如「從錯誤訊息 `CalcTest.java:13` 看到是 `testSubtract`，再去看 `Calc.subtract`……」。
3. 存檔後先在本機確認：

   ```
   git diff
   mvn clean test
   ```

   看到 `Tests run: 14, Failures: 0` 再繼續。
4. commit 並 push：

   ```
   git add src/main
   git commit -m "fix(q2): restore subtract"
   git push
   ```

5. 到 Actions 頁面確認 **CI #3** 變回綠色 ✓，截圖。記下結束時間。

報告 **Q2**「學生修復」可以記錄的項目：

項目	內容
弄壞的方式	`subtract` 的 `a - b` 改成 `a + b`（`7c999df`）
CI 結果	#2 紅燈，`testSubtract`、`testSubtractNegative` 失敗
你的判斷過程	（自己寫）
修法	改程式還是改測試？為什麼？
修復 commit	`fix(q2): ...` 的 hash，CI #3 綠燈
花費時間	從開始到 CI #3 變綠

修好、CI 變綠之後，就進入步驟 **4**：自己選另一種弄壞方式，不要告訴我改了什麼。push 讓 CI 變紅後，傳給我一句請我修的話，附上錯誤訊息。

第 69 則

學生 **prompt**：

〔附圖：CI #2 testSubtractNegative 失敗記錄〕

```
10:00
從錯誤訊息看到是testSubtract出現error 所以先看了testSubtract所在的檔案
CalcTest.java中 testSubtract使用到了什麼函式 看到substract函式後再至calc.java更
改 return a+b to return a-b
```

〔Agent 動作〕檢查學生 repo（最新 commit `015540d fix(q2): restore subtract`，Calc.java 已改回 `a - b`）；查詢 GitHub API，CI #3 completed / success（22:08:29 完成）。

Claude 回應：

修好了，我也確認過 CI 的結果：

項目	結果
修復 commit	`015540d fix(q2): restore subtract`
CI #3	✅ **success**，22:08:29 跑完（台灣時間）
`Calc.java` 第 15 行	`return a - b;` 已改回

花費時間：你寫的開始時間 10:00 應該是晚上 10:00（22:00），所以從開始修到 CI #3 變綠大約 **8** 分半。請把 CI #3 的綠燈截圖補上。

你寫的判斷過程可以直接用，寫進報告時建議修正兩處，讓描述更精確：

1. 失敗的是 **2** 個測試：`testSubtract`（CalcTest.java 第 13 行）和 `testSubtractNegative`（第 18 行）。你補的截圖正好證明了第二個。
2. JUnit 把這種情況算成 **Failure**（斷言不成立），不是 Error（拋出例外）。CI 輸出是 `Failures: 2, Errors: 0`。

參考寫法（用你自己的話改）：

> 從 CI 錯誤訊息看到 `testSubtract`（CalcTest.java:13）和 `testSubtractNegative`（CalcTest.java:18）兩個測試 Failure，訊息都是 "Calc difference incorrect"。我先看 CalcTest.java，發現兩者都呼叫 `Calc.subtract`，再到 Calc.java 檢查 `subtract`，發現被改成 `return a + b`，於是改回 `return a - b`。我修改的是程式，沒有改測試，因為測試的預期值 5−3=2、2−3=−1 是正確的規格。從開始到 CI #3 綠燈約 8 分半。

最後一句「為什麼改程式而不是改測試」很重要，等一下就要拿來和 agent 的做法比較。

另外，你訊息裡附的第二張圖我這邊沒有顯示出來。如果它很重要，請重新傳一次。

接下來是步驟 **4**：交給 **agent** 修。

1. 自己選一種弄壞的方式，不要告訴我。要讓這個比較有意義，最好讓程式和測試看起來都可能是錯的那一方。
2. commit（訊息不要寫出改了什麼，例如 `chore(q2): break the build again`）並 push，讓 CI 變紅。
3. 傳給我一句請我修的話，例如「CI 失敗了，請幫我修好讓 build 通過」，再附上錯誤訊息。

我會像一般的 agent 一樣處理，你觀察我改了什麼、有沒有先問你。

第 70 則

學生 **prompt**：

〔附圖：Actions 列表顯示 CI #3 綠燈、CI #2 testSubtractNegative 失敗記錄〕

`10:08 修好`

Claude 回應：

Q2 你自己修的部分完成了，證據齊全：

項目	證據
弄壞	`7c999df`，CI #2 ❌
被抓到的測試	`testSubtract`（CalcTest.java:13）、`testSubtractNegative`（CalcTest.java:18），兩張截圖都有
修復	`015540d`，CI #3 ✅
花費時間	10:00 → 10:08，約 **8** 分鐘

項目	證據
修的位置	只改了程式(`Calc.java`)，測試沒動

接下來是步驟 **4**：交給 **agent** 修，給我做。

1. 自己選一種弄壞的方式，不要告訴我。想讓比較更有意義的話，可以選「程式和測試看起來都可能是錯的那一方」的改法。例如改 Q6 那些由你決定的規則(modulo 的正負號、`power(0,0)`、負指數)，這樣才看得出 agent 會不會為了讓測試通過，去改掉你定好的規格。

2. commit 並 push。commit 訊息不要寫出改了什麼：

   ```
   git add src
   git commit -m "chore(q2): break the build again"
   git push
   ```

3. 等 CI #4 變紅，記下時間。

4. 傳給我一句請我修的話，例如「CI 失敗了，請幫我修好讓 build 通過」，並附上錯誤訊息的截圖或文字。

收到後，我會像一般的 coding agent 那樣去你的 repo 調查並修復。過程你可以觀察：

- 我改的是程式還是測試
- 修之前有沒有先問你
- 修法跟你的有什麼不同

我修好之後不會自己 commit，會讓你檢查後再 commit、push，然後看 CI #5 有沒有變綠。

第 71 則(Q2：交給 agent 修)

學生 **prompt**：

`CI 失敗了，請幫我修好讓 build 通過`

〔Agent 動作〕1. 檢查學生 repo：`git status`、`git log --oneline -4`、`git show --stat HEAD`、`git show HEAD -- src`，發現最新 commit `3648d1b` 只把 Calc.java 的 multiply 從 `return a * b;` 改成 `return a / b;`。2. 查詢 GitHub API：CI #4(3648d1b)completed / failure。3. 查看 CalcTest.java 中呼叫 `Calc.multiply` 的測試(`15 == Calc.multiply(3, 5)`)。4. 修改學生的 Calc.java，把 multiply 改回 `return a * b;`(未修改測試檔，未 commit)。5. 在沙盒執行測試：OK (14 tests)。

Claude 回應：

修好了，修改留在你的工作區還沒 commit，等你檢查。

我查到什麼

1. GitHub 上 CI #4(commit `3648d1b`)失敗了。

2. `git show HEAD` 顯示這次 commit 只改了 `Calc.java` 的一行：`multiply` 從 `return a * b;` 變成 `return a / b;`。
3. 對應的測試是 `testMultiply`，要求 `multiply(3, 5) == 15`。改壞之後 `3 / 5` 是 0，所以失敗。

我改了什麼

只改 `src/main/java/Calc.java`，把 `multiply` 恢復成原本的寫法，測試沒動：

```
    static public int multiply (int a, int b)
    {
-       return a / b;
+       return a * b;
    }
```

我這邊跑測試的結果是 `OK (14 tests)`。

為什麼改程式而不是改測試：`3 × 5 = 15` 是正確的乘法規格，錯的是程式。而且這一行是你在 Q1 commit `75093c5` 裡寫的，最近一次 commit 是唯一動過它的地方。

請你檢查後 **commit**：

```
git diff
mvn clean test
git add src/main
git commit -m "fix(q2): restore multiply (agent fix)"
git push
```

push 之後 CI #5 應該會變綠。

寫進 **Q2** 比較時要誠實說明的一點：我沒有讀 CI 的錯誤訊息，而是直接看了最新 **commit** 的 **diff**。這是 coding agent 常見的捷徑，但如果錯誤是好幾個 commit 以前引入的，這招就找不到。你是從錯誤訊息開始往回追，路徑不一樣。

	你（CI #2→#3）	agent（CI #4→#5）
從哪裡開始	CI 錯誤訊息 → 測試 → 程式	最新 commit 的 diff
改了什麼	程式	程式
有沒有改測試	沒有	沒有
修之前有沒有問你	—	沒有問，直接修改工作區，但沒有自己 commit
時間	約 8 分鐘	約 1 分鐘

這次弄壞的地方很明顯，所以兩種做法結果一樣。你可以在報告裡說明：這種情況下 agent 比較快，但它依賴 diff，而且沒有確認「測試會不會才是錯的」。

第 72 則

學生 **prompt**:

我想寫 這次弄壞的部分很顯而易見，也不需要追溯到好幾個檔案，這種情況下agent較快判斷出錯誤，但依賴diff，若是多個commit以前的，這個方式就找不到錯誤，並且agent也沒有確認test是否有錯誤

如果以上是我想寫的部分 我把我的report檔案如何給你 你可以直接幫我把以上截圖和我需要寫的部分都打上去

Claude 回應:

可以，直接修改你的報告檔我能做到，有兩種方式把檔案交給我:

1. 放進 **`calc-tdd`** 資料夾（建議）:這個資料夾我本來就能讀寫，你把報告複製進去，告訴我檔名就好。改完會直接存回同一個位置，原檔我會先另存一份備份。
2. 直接傳到對話裡:我改完再傳回給你，你下載後覆蓋原檔。

請給我 **Word** 檔（**.docx**），不要給 **PDF**。PDF 幾乎沒辦法直接修改，Word 改完你可以自己再匯出成 PDF。

我會加上的內容:

- **Q2**:
  - 你修的部分:CI #2 紅燈列表、`testSubtract` 和 `testSubtractNegative` 的失敗訊息、CI #3 綠燈截圖，以及你寫的判斷過程和 10:00→10:08 的時間。
  - agent 修的部分:CI #4 紅燈、我的 diff，以及你剛才寫的比較文字，一字不改。
- **Q3**:SpotBugs 和 JaCoCo 的 CI 記錄（文字＋截圖）、JaCoCo 報告表格，以及 `power` 原始碼的覆蓋率截圖。

還缺一張圖:**CI #5** 的綠燈。請先把我修的部分 commit 並 push:

```
git add src/main
git commit -m "fix(q2): restore multiply (agent fix)"
git push
```

等 CI #5 變綠後截圖給我，CI #4 紅燈的頁面也一起截。

你寫的文字有一句建議修改，免得跟對話紀錄對不上:「agent 也沒有確認 test 是否有錯誤」這句不完全正確。我有看 `testMultiply` 並說明「3 × 5 = 15 是正確的規格，所以改程式」，但這是我自己判斷的，沒有問你。比較準確的寫法是:

> agent 雖然有說明為什麼改程式而不是改測試，但這是它自己判斷的，沒有先詢問我測試是否才是錯的那一方。

你決定要不要換成這句，我會照你最後給的版本放進報告。

第 73 則

學生 **prompt**:

〔附圖：Actions 列表顯示 CI #5 綠燈、CI #4 紅燈〕〔附檔：REPORT.md.docx〕

agent 雖然有說明為什麼改程式而不是改測試，但這是它自己判斷的，沒有先詢問我測試是否才是錯的那一方。
改成以上這句

〔Agent 動作〕讀取上傳的 REPORT.md.docx；查詢 GitHub API 取得 CI #1–#5 的時間與結果、學生 repo 的最新 git log；在 docx 中以 XML 編輯方式：把 §3.2、§3.3 的空白欄位換成 Q2、Q3 內容並插入 11 張學生提供的截圖；填入 Repo 網址與期間；更新繳交清單、§1.1 與附錄 D 的截圖引用、附錄 C 的 git log、§5 AI 使用紀錄表；以 Word 格式驗證並轉成 PDF 檢查排版；把新檔傳給學生（原檔未改動）。

Claude 回應：

我已經在你的報告裡補上 Q2 和 Q3，並另存一份新檔傳給你。你原本的檔案沒有被改動，請下載後自行取代。檔案已通過 Word 格式檢查，我也轉成 PDF 逐頁看過排版。

§3.2 Q2

- 一張 CI #1 到 #5 的時間表，commit 和完成時間都是從 GitHub 實際查到的。
- 你自己修的部分：一張事實整理表；「我的判斷過程」是你寫的原文，一字未改；附 4 張截圖。
- **agent** 修的部分：
  - 給我的那句 prompt，逐字照錄。
  - 我怎麼調查：直接看最新 commit 的差異，沒有讀 CI 錯誤訊息。
  - 我改的那一行，以及 CI #5 的綠燈截圖。
- 比較：一張比較表，下面接你寫的那段文字，已換成你指定的那句。

§3.3 Q3

- JaCoCo 和 SpotBugs 的 CI 紀錄，貼成文字並附截圖。
- 兩張 JaCoCo 報告表格、`power` 的原始碼覆蓋截圖，再加一張指標解讀表。

順便修正的地方

- 開頭填上 Repo 網址，期間改成到 10-25。
- 繳交清單裡 Q2、Q3 標成已完成。
- §1.1 和附錄 D 原本指向已刪掉的 `q1-screenshots/`，改成指向 §1.3。
- 附錄 C 的 git log 補上 5 個新 commit。
- §5 AI 使用紀錄新增 Q2 一列，「報告」那列註明 §3.2 的判斷過程和比較文字是你寫的。

還有一處要你自己處理：**§2.6「Q1 與 Q6 比較」**還是我當初留的提示文字，開頭是「可以比較的面向……」，這段需要換成你自己的比較。

全部完成後，最後一步是匯出這整段對話，當作 AI 使用紀錄。

第 74 則

學生 **prompt**:

如何匯出

〔Agent 動作〕嘗試以網路搜尋查詢官方說明(組織已停用搜尋功能, 失敗)。

Claude 回應:

我這邊查不到官方說明文件(網路搜尋在你的組織被停用了), 所以下面是依我所知的做法, 實際按鈕位置請以你的 **app** 畫面為準。

方法一:官方資料匯出(最正式)

1. 打開 Claude 網頁版 claude.ai, 或桌面 app 左下角的頭像 → **Settings**。
2. 進到 **Privacy**, 找 **Export data**。
3. 按下後, 系統會寄一封 email 給你, 裡面有下載連結。下載到的 zip 裡包含所有對話的完整紀錄(JSON 格式)。

注意:你的帳號屬於組織(NYCU ATLAB), 組織版有時只有管理員能匯出。如果你看不到這個按鈕, 就是這個原因, 改用方法二。

方法二:手動複製(最保險、評分最好讀)

1. 在這個對話裡從第一則訊息開始, 把每一則你的 prompt 和我的完整回應依序複製, 貼進一份 Word 文件。
2. 每則前面標「學生 **prompt**:」或「**Claude** 回應:」。
3. 你傳過的截圖在報告裡已經有了, 這份紀錄中可以寫「(附截圖, 見報告 §X)」代替。
4. 存成 PDF, 例如 `AI-usage-log.pdf`, 和報告一起交。

對話很長, 但這樣做最能證明內容是「逐字」的。

方法三:截圖整段對話

如果時間不夠, 可以從上到下捲動截圖存成 PDF。不過文字比截圖好讀, AI 評分時也比較準確。

不建議由我來產生紀錄。我可以根據對話內容寫一份逐字稿, 但由 AI 自己整理的「AI 使用紀錄」, 可信度比不上官方匯出或你親手複製的版本。如果方法一、二都不可行, 再告訴我, 我可以產生一份當作備用, 並在檔案開頭清楚標明「由 Claude 依對話內容整理」。

匯出後, 記得在報告 §5 把「對話匯出檔(附件)」改成你實際的檔名。

第 75 則

學生 **prompt**:

內容太長 麻煩你逐字的以方法二整理一份word 檔 並在最前面同步說明因為是團體帳號所以無法自行匯出

〔Agent 動作〕依對話內容逐字整理本文件（AI-usage-log.docx），並在開頭加上說明。

Claude 回應：

（即本文件。Claude 交付本檔時的說明訊息未收錄於本檔內。）

附錄：Q7 全新 sub-agent 的逐字紀錄

此 sub-agent 由 Claude（Cowork）在第 54 則的處理過程中啟動，沒有本對話的任何脈絡。以下 prompt 與回應皆為逐字照錄（另存於 repo 的 `q7/agent-transcript.md`）。

- Agent: Claude (sub-agent started by the Cowork session), fresh context: it had never seen this conversation, `Calc.java`, or the Q6 history.
- Date: 2026-10-23 (Asia/Taipei)
- Input: only `CalcTest.java` (the 14 tests from Q1 + Q6) with all `//` comments removed, so the agent could not read the intended rules from comments.
- Tools used by the agent: none (tool_uses: 0).

Prompt（逐字）

````
Write a Java class `Calc` (file Calc.java, default package) so that every
test in the JUnit 4 test class below passes. Do not modify the tests. Do not
use any tools and do not look at any files — work only from the test code in
this message. Reply with the complete Calc.java source in one code block,
followed by a short list of any behaviour you had to decide that the tests do
not pin down.

```java
import org.junit.*;
import static org.junit.Assert.*;

public class CalcTest
{
   @Test public void testAdd()
   {
      assertTrue ("Calc sum incorrect", 5 == Calc.add (2, 3));
   }

   @Test public void testSubtract()
   {
      assertTrue ("Calc difference incorrect", 2 == Calc.subtract ( 5, 3));
   }

   @Test public void testSubtractNegative()
   {
      assertTrue ("Calc difference incorrect", -1 == Calc.subtract ( 2, 3));
   }

   @Test public void testMultiply()
````
````

```
   {
      assertTrue ("Calc product incorrect", 15 == Calc.multiply ( 3, 5));
   }

   @Test public void testDivide()
   {
      int result = Calc.divide(7, 2);
      assertTrue ("Calc quotient incorrect", 3 == result);
   }

   @Test(expected = ArithmeticException.class)
   public void testDivideByZero()
   {
      Calc.divide(7, 0);
   }

   @Test public void testDivideNegtive()
   {
      int result = Calc.divide(7, -2);
      assertTrue ("Calc quotient incorrect", -3 == result);
   }

   @Test(expected = ArithmeticException.class)
   public void testModuloByZero()
   {
      Calc.modulo(7, 0);
   }

   @Test public void testModuloNegativeOperands()
   {
      assertTrue ("Calc remainder incorrect for (-7, 3)", -1 == Calc.modulo
(-7, 3));
      assertTrue ("Calc remainder incorrect for (7, -3)",  1 == Calc.modulo
(7, -3));
      assertTrue ("Calc remainder incorrect for (-7, -3)", -1 == Calc.modulo
(-7, -3));
   }

   @Test public void testPowerZeroToZero()
   {
      assertTrue ("Calc power incorrect for (0, 0)", 1 == Calc.power (0, 0));
   }

   @Test public void testPowerPositiveExponent()
   {
      assertTrue ("Calc power incorrect for (2, 3)", 8 == Calc.power (2, 3));
   }

   @Test public void testPowerNegativeExponentTruncates()
```

```
   {
      assertTrue ("Calc power incorrect for (2, -1)",   0 == Calc.power (2,
-1));
      assertTrue ("Calc power incorrect for (-2, -1)",  0 == Calc.power (-2,
-1));
      assertTrue ("Calc power incorrect for (1, -3)",   1 == Calc.power (1,
-3));
      assertTrue ("Calc power incorrect for (-1, -3)", -1 == Calc.power (-1,
-3));
      assertTrue ("Calc power incorrect for (-1, -2)",  1 == Calc.power (-1,
-2));
   }

   @Test(expected = ArithmeticException.class)
   public void testPowerZeroToNegativeThrows()
   {
      Calc.power(0, -1);
   }

   @Test(expected = ArithmeticException.class)
   public void testPowerOverflowThrows()
   {
      Calc.power(2, 31);
   }
}
```

Sub-agent 回應（逐字、完整）

```
public class Calc
{
   public static int add(int a, int b)
   {
      return a + b;
   }

   public static int subtract(int a, int b)
   {
      return a - b;
   }

   public static int multiply(int a, int b)
   {
      return a * b;
   }

   /** Integer division that truncates toward zero. Throws
ArithmeticException if b == 0. */
   public static int divide(int a, int b)
   {
```

```java
      return a / b;
   }

   /** Remainder whose sign follows the dividend, like Java's %. Throws
ArithmeticException if b == 0. */
   public static int modulo(int a, int b)
   {
      return a % b;
   }

   /**
    * Integer power.
    *  - power(0, 0) == 1
    *  - Negative exponent: the exact result 1 / base^|exp| is truncated
toward zero,
    *    so it is 1 for base 1, +/-1 for base -1 and 0 for any other non-zero
base.
    *  - power(0, negative) throws ArithmeticException (division by zero).
    *  - Throws ArithmeticException if the result does not fit in an int.
    */
   public static int power(int base, int exp)
   {
      boolean oddExp = (exp & 1) != 0;   // also correct for negative
exponents

      if (exp < 0)
      {
         if (base == 0)
            throw new ArithmeticException("0 raised to a negative power");
         if (base == 1)
            return 1;
         if (base == -1)
            return oddExp ? -1 : 1;
         return 0;
      }

      if (exp == 0)
         return 1;

      // Bases whose powers never overflow; handling them here keeps large
exponents fast.
      if (base == 0 || base == 1)
         return base;
      if (base == -1)
         return oddExp ? -1 : 1;

      // When |base| >= 2 this loop overflows within about 31 steps, so it
stays short.
      int result = 1;
```

```
        for (int i = 0; i < exp; i++)
            result = Math.multiplyExact(result, base);   // throws
ArithmeticException on overflow
        return result;
    }
}
```

Choices the tests don't settle:

- **Overflow in `add`, `subtract`, `multiply` and `divide`:** these use plain Java `int` arithmetic, so an overflow wraps around without an error. For example, `divide(Integer.MIN_VALUE, -1)` returns `Integer.MIN_VALUE`. Only `power` throws on overflow, because only that one is tested for it. If you want all of them to behave the same way, use `Math.addExact`, `Math.subtractExact` and `Math.multiplyExact` instead.
- **Rounding and signs:** `divide` rounds toward zero. `modulo` is Java's `%`, so the result takes the sign of the first number. That matches the three negative-number cases in the tests, but it is not mathematical (always non-negative) modulo.
- **Return type:** every method takes and returns `int`.
- **Negative exponents:** for bases other than 1 and -1, `power` returns 0. For base -1 it returns ±1 depending on whether the exponent is odd or even. The tests check these at -1, -2 and -3 only; I applied the same rule to every negative exponent.
- **`power(Integer.MIN_VALUE, 1)`:** returns `Integer.MIN_VALUE` without throwing. `power(Integer.MIN_VALUE, 2)` does throw.
- **Large exponents:** bases 0, 1 and -1 are handled directly, so something like `power(1, Integer.MAX_VALUE)` returns at once instead of running a long loop.